



Internship Report On

“DIGITALIZATION OF ACCEPTANCE USAGE POLICY(AUP) & PASSWORD POLICY.”

Submitted By: GADE LOHITHA REDDY

Under the guidance of: Mr. Arunanjan Pattanayak

Unit IT-Head Information System Department
ITC Ltd-PSPD Unit, Bhadrachalam.



IT Department

This is to certify that the project report entitled “**Digitalization of Acceptance Usage Policy and Password Policy**”, is a Bonafide record of work done by **Mrs. Gade Lohitha Reddy** Under supervision in IT Dept at ITC PSPD ltd.

Unit IT Head

(Mr. Arunanjan Pattanayak)

Company details:

ITC Limited
PAPERBOARDS & SPECIALITY
PAPERS DIVISION SARAPAKA -
507128
Bhadradri kothagudem Dist., Telangana

ACKNOWLEDGEMENT

I thank the people concerned with the management at ITC-PSPD BCM for providing me with this opportunity to undergo internship training at the IS department.

I would now bound to thank **Mr. Sourav Mukherjee** (HR Department), **Mr. Arunanjan Pattanayak** (Unit IT Head) for granting me permission and explore my internship in such a great platform.

Secondly, I thank all the staff of ITC Ltd-PSPD, Especially **Mr. V. Sai Ram** (IS Department) who helped me in getting the programming done in IS department, ITC Ltd- PSPD. I'm very grateful to all who have extended their kind cooperation in Information System Department.

I thank each and every one who helped me in completing the internship.

CONTENTS

Contents
Introduction
Overview of Organization
Overview of the Project
Technologies Used
Database Schema
System Implementation
System Architecture and workflow
Code
Explanation of each part of code
Screenshots
Conclusion

INTRODUCTION

The internship marked my first meaningful step into the professional world, allowing me to connect the theoretical knowledge I had acquired with practical, real-life applications. This one-month training experience provided me with valuable exposure that will undoubtedly support my future career endeavors.

My primary motivation for joining a reputable organization as an intern was to observe and understand how professionals interact and operate in a workplace setting. Through this opportunity, I learned how to collaborate effectively with colleagues, communicate confidently, and approach challenges with a refreshed and structured problem-solving mindset. This experience has significantly shaped my outlook and strengthened my professional development.

This report reflects my observations and learnings throughout the internship period. It instilled in me a strong sense of responsibility, the importance of teamwork, and the ability to adapt to and resolve real-world issues. While my tasks as an intern were foundational, the exposure to a functioning work environment gave me a glimpse of what practical life entails.

I also gained insight into how information seamlessly flows across departments using digital tools and networking systems. Additionally, I improved my communication skills by interacting with team members and responding to queries.

The specific objectives of the study are:

- To implement the theoretical knowledge in the real-world environment.
- To build up confidence, interpersonal and communication skills.
- To understand how resource sharing is done in a real working environment.
- To gain knowledge about networking and its various technical terms.
- To know how documentation is done, records are created and maintained and security consideration is implemented.
- To increase a sense of responsibility and acquire good work habit.

OVERVIEW OF ORGANIZATION

ITC is one of India's foremost multi-business enterprises with a market capitalization of US\$45 billion and a turnover of US\$ 8 billion. ITC is rated among the world's best big companies,

Asia's 'Fab 50' and the world's Most Reputable Companies by Forbes magazine and Hay Group. ITC also features as one of world's largest sustainable value creator in the consumer goods industry in a study by the Boston Consulting Group. ITC has been listed among India's '10 Most Valuable (Company) Brands', according to a study conducted by Brand Finance and published by the Economic Times.

ITC also ranks among Asia's 50 best performing companies compiled by Business week. ITC is India's leading Fast Moving Consumer Goods company, the clear market leader in the Indian Paperboard and Packaging industry a globally acknowledged pioneer in farmer empowerment through its wide-reaching Agri Business and runs the greenest luxury hotel chain in the world. ITC info Tech, a wholly-owned subsidiary, is one of India's fast-growing IT companies in the mid- tier segment. This portfolio of rapidly growing business considerably enhances ITC's capacity to generate growing value for the Indian economy. ITC PSPD, Bhadrachalam consists of the following:

- 7 machines with production capacity of 412,000 TPA of Paperboard and 140,000 TPA of Paper.
- Specialty boards for playing cards and scratch cards, C2S Art Boards and Ivory Boards
- Commissioned India's first Elemental Chlorine Free bleaching line, later followed by another line incorporating Light ECF process with a combined capacity of 260,000 TPA



OVERVIEW OF THE PROJECT

The Digitalization of Acceptable Use Policy (AUP) and Password Policy Management System is a web-based application developed to automate and streamline the process of employee registration, authentication, policy acknowledgment, and administrative monitoring within an organization. The project aims to replace traditional paper-based and manual processes with a secure digital platform that improves efficiency, accuracy, and record management.

The system provides a centralized environment where employees can register their details, complete authentication procedures, and digitally acknowledge organizational policies. The registration process includes employee information validation, email verification, and secure authentication mechanisms to ensure that only authorized users gain access to the system. By integrating these features, the application enhances security while reducing administrative effort and manual documentation.

A key feature of the project is the implementation of security-oriented authentication methods such as email verification, One-Time Password (OTP) validation, and face authentication. These mechanisms help verify user identity and strengthen access control. The system also enforces password policy requirements, encouraging users to create strong and secure passwords in accordance with organizational standards.

The application includes an administrative dashboard that enables administrators to manage employee records, monitor registration activities, verify policy acknowledgments, and maintain system data efficiently. All user information and authentication records are stored digitally, allowing quick retrieval, easy management, and improved transparency in organizational operations.

The project was developed using Python and the Flask framework for backend processing, along with HTML, CSS, and JavaScript for the user interface.

The system focuses on security, usability, and automation, providing a reliable solution for managing employee authentication and policy compliance. Through digital transformation of these processes, the project contributes to improved operational efficiency, enhanced security, and better record management within the organization.

The application also promotes accountability by maintaining digital records of user actions and policy acknowledgments. Employees are required to review and accept organizational policies before completing the registration process, ensuring that policy compliance is properly documented. This digital approach provides a reliable mechanism for tracking employee awareness and acceptance of organizational guidelines.

Overall, the project demonstrates the practical application of web development, database management, authentication mechanisms, and security principles to solve a real-world organizational problem. By combining automation, digital record management, and secure authentication techniques, the system provides an effective solution for managing employee registration and policy compliance in a modern digital environment.

TECHNOLOGIES USED

Python

Python serves as the core programming language for the development of the Digitalization of AUP & Password Policy System. It is responsible for implementing the application logic, processing user requests, validating input data, managing authentication processes, handling database operations, and integrating external services such as email and SMS notifications. Python's simplicity and extensive library support make it suitable for developing secure and scalable web applications.

Flask Framework

Flask is a lightweight web framework used to build the backend of the application. It manages URL routing, request handling, session management, page rendering, and communication between the frontend and the database. Flask enables the creation of dynamic web pages and provides the foundation for the entire application workflow.

HTML

HTML (HyperText Markup Language) is used to create the structure and content of web pages within the system. Pages such as registration forms, login interfaces, policy acknowledgment screens, authentication pages, and the administrative dashboard are developed using HTML. It forms the basic framework of the user interface.

CSS

CSS (Cascading Style Sheets) is used to improve the visual appearance and layout of the application. It controls colors, fonts, spacing, alignment, responsiveness, and overall page design. CSS helps provide a professional interface for users.

JavaScript

JavaScript is used to implement client-side functionality and improve user interaction. It enables dynamic page behavior, form validation, camera integration, asynchronous communication with the server.

SQLite Database

SQLite is the database management system used in the project. It stores employee registration details, contact information, authentication records, uploaded image paths, and submission timestamps. SQLite provides a lightweight and efficient database solution that integrates seamlessly with Flask applications.

Face Recognition Library

The Face Recognition library is used to implement biometric authentication within the system. It detects facial features, generates facial encodings, and compares captured facial images with stored employee images. This technology enables secure identity verification and strengthens access control within the application.

SMTP (Simple Mail Transfer Protocol)

SMTP is used for sending automated emails to registered employees. The system uses SMTP services to deliver registration confirmations, policy notifications, and other important communications. This ensures reliable and secure email-based communication between the system and users.

Gmail Email Service

The Gmail email service is integrated into the application for sending automated emails. Gmail App Password authentication is used to securely connect the application with the email server, ensuring safe transmission of registration and policy-related notifications.

Requests Library

The Requests library is used to communicate with external web services. In this project, it is utilized for sending SMS notifications through an online SMS gateway. This allows employees to receive registration confirmation messages directly on their mobile devices.

Regular Expressions (Regex)

Regular expressions are used for input validation throughout the application. Employee IDs, names, phone numbers, and email addresses are verified using predefined patterns to ensure that only valid and correctly formatted information is accepted by the system.

Session Management

Session management is implemented using Flask sessions to maintain administrator login status and control access to protected pages. This mechanism helps secure administrative functions and prevents unauthorized access to sensitive information.

Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for coding, debugging, testing, and maintaining the application. It provides an efficient workspace with tools and extensions that enhance productivity during development.

Web Browser

Modern web browsers such as Google Chrome and Microsoft Edge are used to access and test the application. They provide the environment required for displaying web pages, executing JavaScript code, capturing images through the webcam, and interacting with the system.

DATABASE SCHEMA

The system uses an SQLite database named employee. dB to store and manage employee registration information. The database serves as the central repository for maintaining employee details, authentication records, contact information, uploaded photographs, and registration timestamps. The database design ensures efficient storage, retrieval, and management of employee data while supporting authentication and administrative operations.

Employees Table

The application contains a table named **employees**, which stores all employee registration details submitted through the system.

Table Structure

Field Name	Data Type	Description
id	INTEGER	Unique identifier for each employee record (Primary Key)
employee_id	TEXT	Employee identification number
name	TEXT	Employee full name
jh_name	TEXT	Job Holder (JH) name
dmt_name	TEXT	Department Manager Team (DMT) name
phone	TEXT	Employee mobile number
email	TEXT	Employee email address
photo_path	TEXT	Location of the stored employee photograph
submission_time	TEXT	Date and time of registration submission

SQL Table Creation

```
CREATE TABLE IF NOT EXISTS employees (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    employee_id TEXT,  
    name TEXT,  
    jh_name TEXT,  
    dmt_name TEXT,  
    phone TEXT,  
    email TEXT,  
    photo_path TEXT,  
    submission_time TEXT  
);
```

Database Operations

The database performs several operations throughout the application lifecycle:

Record Insertion

When an employee successfully completes registration, the entered information is stored in the employees table along with the captured facial image path and registration timestamp.

Record Retrieval

Employee records are retrieved during face authentication, profile verification, and administrative monitoring activities.

Record Update

When an employee re-registers or updates their profile, the system replaces the previous photograph with the latest captured image and updates the corresponding database record.

Record Deletion

To prevent duplicate entries, old employee records associated with the same Employee ID are removed before storing updated information.

Database Workflow

Employee Registration → Data Validation → Database Storage → Face Authentication → Record Retrieval → Administrative Monitoring

The database acts as the backbone of the system by maintaining accurate employee information and supporting secure authentication processes throughout the application.

EMPLOYEES REGISTRATION DETAILS

employee_id
name
jh_name
dmt_name
phone
email
photo_path
submission_time

SYSTEM IMPLEMENTATION

The Digitalization of AUP & Password Policy System was implemented as a web-based application using Python and the Flask framework. The system integrates employee registration, face authentication, email notifications, SMS communication, database management, and administrative monitoring into a single platform. The implementation focuses on improving security, reducing manual effort, and ensuring efficient management of employee records and policy compliance.

Employee Registration Module

The employee registration module was developed to collect employee information such as Employee ID, Name, JH/DMT details, Phone Number, and Email Address. The system validates all user inputs before storing them in the database. A photograph of the employee is captured through the webcam and stored securely for future authentication purposes.

Data Validation Implementation

Input validation mechanisms were implemented using regular expressions and server-side checks. The system verifies employee IDs, names, phone numbers, and email addresses before accepting registration requests. Invalid or incomplete data is rejected to maintain database integrity and prevent unauthorized entries.

Face Authentication Implementation

The face authentication feature was implemented using the Face Recognition library. During registration, employee photographs are captured and stored in the system. During login or verification, a live image is captured and compared with stored facial data. Authentication is granted only when a valid facial match is identified.

Database Implementation

SQLite was used as the backend database to store employee information and authentication-related data. The database maintains employee details, image paths, contact information, and registration timestamps. Database operations such as insertion, retrieval, updating, and deletion are performed using SQL queries integrated within the Flask application.

Email Notification Implementation

An automated email notification system was implemented using SMTP services. Upon successful registration, confirmation emails are automatically sent to employees. The system also sends policy-related notifications containing quarterly policy updates and important security instructions.

SMS Notification Implementation

SMS functionality was integrated using an online SMS gateway service. After successful registration, employees receive confirmation messages on their registered mobile numbers. This ensures immediate communication and provides an additional notification channel.

Administrative Dashboard Implementation

An administrator dashboard was developed to provide centralized control over employee records. Authorized administrators can log in securely, view employee information, monitor registrations, and manage stored records. Session management techniques are used to restrict dashboard access to authorized users only.

Session and Access Control Implementation

Flask session management was implemented to maintain secure administrator login sessions. Access to protected pages is restricted unless valid authentication credentials are provided. This mechanism helps prevent unauthorized access to administrative functions.

File Management Implementation

The system automatically manages employee photographs by storing them within a designated directory structure. When updated photographs are submitted, older images associated with the employee are removed to avoid duplication and unnecessary storage usage.

User Interface Implementation

The user interface was developed using HTML, CSS, and JavaScript. The design provides simple navigation, responsive layouts, and interactive components that enhance user experience. Forms, authentication screens, dashboards, and policy pages are designed to ensure ease of use and accessibility.

Security Implementation

Multiple security measures were incorporated into the system, including face authentication, input validation, session management, administrator access control, email verification, and secure communication mechanisms. These features collectively improve system reliability and protect sensitive employee information.

Overall Implementation Process

The implementation process follows a structured sequence beginning with employee registration, followed by data validation, image capture, database storage, email and SMS notification generation, face authentication, and administrative monitoring. This integrated approach ensures secure employee management and effective policy compliance within the organization.

SYSTEM ARCHITECTURE AND WORKFLOW

The Digitalization of AUP & Password Policy System follows a web-based client-server architecture. The system consists of four major components: the User Interface, Flask Application Server, SQLite Database, and External Communication Services. These components work together to provide secure employee registration, authentication, policy management, and administrative monitoring.

The User Interface serves as the interaction layer where employees and administrators access the system through web browsers. Employees can register, submit details, capture photographs, and perform authentication, while administrators can monitor and manage employee records through the dashboard.

The Flask Application Server acts as the core processing unit of the system. It receives requests from users, validates inputs, performs authentication operations, processes business logic, communicates with the database, and generates appropriate responses. The server is also responsible for handling email notifications, SMS communication, session management, and face verification operations.

The SQLite Database functions as the data storage component of the system. It securely stores employee details, contact information, image paths, registration timestamps, and other records required for system operation. The database supports data retrieval and management activities performed by both employees and administrators.

External Communication Services are integrated to support email and SMS notifications. SMTP services are used to send registration confirmation emails and policy updates, while SMS gateway services deliver registration confirmation messages to employees.

The authentication layer is responsible for verifying the identity of employees before allowing access to the system. This layer includes OTP verification, face image capture, and facial authentication mechanisms. By combining multiple verification techniques, the system provides enhanced security and minimizes the possibility of unauthorized registrations. The authentication process ensures that only genuine employees can complete the policy acceptance and registration procedures.

The database layer is responsible for storing and managing all system data. SQLite is used as the backend database to maintain employee information, policy acceptance records, feedback details, authentication data, and registration logs. The database enables efficient storage, retrieval, and updating of records while maintaining data consistency and integrity throughout the system.

The system also incorporates a policy management component that displays the latest AUP and Password Policy versions to employees. Before completing registration, employees must review and accept the mandatory policies. This ensures compliance with organizational security standards and maintains a digital record of policy acceptance for future reference and auditing purposes.

Another important component of the architecture is the feedback management module. This module collects user feedback regarding the registration process, allowing administrators to evaluate user satisfaction and identify areas for improvement. The collected feedback supports continuous enhancement of the system and improves the overall user experience.

System Workflow

The workflow begins when an employee accesses the registration page and enters the required personal and organizational information. The system validates all entered details to ensure data accuracy and completeness.

After successful validation, the employee's photograph is captured through the webcam and stored securely. The employee information and image path are then saved in the SQLite database.

Once registration is completed, the system automatically generates confirmation notifications. A registration confirmation email and policy update email are sent to the employee's registered email address. Additionally, an SMS notification is sent to the registered mobile number.

When an employee attempts authentication, the system captures a live facial image and compares it with the stored image using the face recognition module. If the facial match is successful, access is granted; otherwise, the authentication request is rejected.

Administrators can access the dashboard through a secure login mechanism. The dashboard retrieves employee records from the database and displays them for monitoring and management purposes.

All user information, facial data, and attendance records are maintained in an SQLite database. The database supports efficient storage, retrieval, and management of data, allowing the system to perform authentication and attendance operations quickly and accurately. Administrators can access attendance records and user information through the management dashboard for monitoring and reporting purposes.

Finally, when the user completes the session, the logout process securely terminates the active session and prevents unauthorized access to the system.

CODE

admin_dashboard.html

```
<!DOCTYPE html>

<html>

<head>

<title>Admin Dashboard</title>

<style>

  body{

    font-family:sans-serif;

    background:#f1f5f9;

    padding:40px;

  }

  .header{

    display:flex;

    justify-content:space-between;

    align-items:center;

    margin-bottom:20px;

  }

  .logo-section {

    text-align:center;

    margin-bottom:25px;

  }

  .logo-section img{
```

```
width:90px;
height:90px;
object-fit:contain;
border-radius:10px;
background:white;
padding:5px;
}
```

```
. logo-section h1{
margin-bottom:5px;
color: #0f172a;
}
```

```
. logo-section p {
margin-top:0;
color:gray;
}
```

```
table {
width:100%;
border-collapse:collapse;
background:white;
border-radius:10px;
overflow:hidden;
box-shadow:0 4px 6px rgba(0,0,0,0.05);
```

```
}
```

```
th, td {  
    padding:15px;  
    text-align:left;  
    border-bottom:1px solid #e2e8f0;  
}
```

```
th{  
    background: #0f172a;  
    color:white;  
}
```

```
img{  
    width:50px;  
    height:50px;  
    border-radius:50%;  
    object-fit:cover;  
}
```

```
.logout{  
    color:#ef4444;  
    text-decoration:none;  
    font-weight:bold;  
}
```



```
.btn-container {  
    margin-top:25px;  
    display:flex;  
    gap:20px;  
    align-items:center;  
    flex-wrap:wrap;  
    margin-bottom:20px;  
}
```

```
select {  
    padding:10px;  
    border-radius:6px;  
    border:1px solid #ccc;  
}
```

```
.print-btn{  
    padding:12px 25px;  
    background: #0074D9;  
    color:white;  
    border:none;
```

```
border-radius:8px;  
    cursor:pointer;  
    font-weight:bold;  
    font-size:16px;
```

```
}
```

```
.print-btn:hover{  
    background: #005fa3;  
}
```

```
.no-data {  
    margin-top:20px;  
    font-weight:bold;  
    color:red;  
    display:none;  
}
```

```
. stats {  
    margin-bottom:20px;  
    display:flex;  
    gap:20px;  
}
```

```
. card {  
    padding:10px 15px;  
    background:white;  
    border-radius:8px;  
    box-shadow:0 2px 5px rgba(0,0,0,0.05);
```

```
}
```

```
/* Interactive Feedback Badges Styles */
```

```
. rating-badge {
```

```
    display: inline-block;
```

```
    padding: 5px 10px;
```

```
    border-radius: 6px;
```

```
    font-size: 12px;
```

```
    font-weight: bold;
```

```
}
```

```
. rating-excellent {background: #d1fae5; color: #065f46; }
```

```
. rating-very-good {background: #dbeafe; color: #1e40af; }
```

```
. rating-good {background: #fef3c7; color: #92400e; }
```

```
. rating-average {background: #ffedd5; color: #9a3412; }
```

```
. rating-poor {background: #fee2e2; color: #991b1b; }
```

```
. comment-text {
```

```
    max-width: 200px;
```

```
    word-wrap: break-word;
```

```
    font-size: 13px;
```

```
    color: #475569;
```

```
    font-style: italic;
```

```
}
```

```
@media print {
```

```
.logout,  
.btn-container,  
. stats{  
    display:none;  
}  
  
body {  
    padding:0;  
    background:white;  
}  
  
table {  
    box-shadow:none;  
}  
}  
</style>
```

```
</head>
```

```
<body>
```

```
<div class="logo-section">
```

```

```

<h1>Admin Dashboard</h1>

<p>

AUP & Password Policy Digitalization System

</p>

</div>

<div class="header">

<div></div>

Logout

</div>

<div class="stats">

<div class="card">

Total Employees: <b id="totalCount">

</div>

<div class="card">

Visible: <b id="visibleCount">

</div>

<div class="card" style="border-left: 4px solid #ef4444;">

Attention Required (Poor/Average Ratings): <b
id="attentionCount">0

</div>

</div>

<div class="btn-container">

<input

type="text"

```
id="searchInput"
placeholder="Search by Name / ID / Phone / Feedback"
onkeyup="searchTable()"
style="
    padding:10px;
    width:300px;
    border-radius:6px;
    border:1px solid #ccc;
">
```

```
<select id="filter" onchange="filterTable()">
    <option value="ALL">All Employees</option>
    <optgroup label="JH CATEGORY">
        <option value="JH ISI">JH ISI</option>

        <option value="JH HR">JH HR</option>
        <option value="JH IT">JH IT</option>
        <option value="JH 1A">JH 1A</option>
        <option value="JH Department">JH Department</option>
        <option value="JH OTHERS">JH OTHERS</option>
    </optgroup>
```

```
    <optgroup label="DMT CATEGORY">
        <option value="DMT VETEND">DMT VETEND</option>
        <option value="DMT SEMISKILLED">DMT
SEMISKILLED</option>
```

```
<option value="DMT TIME OFFICE">DMT TIME  
OFFICE</option>
```

```
<option value="DMT TRAINEE">DMT TRAINEE</option>
```

```
<option value="DMT MANAGER">DMT MANAGER</option>
```

```
</optgroup>
```

```
</select>
```

```
<select id="ratingFilter" onchange="filterTable()">
```

```
<option value="ALL">All Ratings</option>
```

```
<option value="Excellent">Excellent</option>
```

```
<option value="Very Good">Very Good</option>
```

```
<option value="Good">Good</option>
```

```
<option value="Average">Average</option>
```

```
<option value="Poor">Poor</option>
```

```
</select>
```

```
<button class="print-btn" onclick="window.print()">
```

```
Print Selected
```

```
</button>
```

```
</div>
```

```
<table>
```

```
<tr>
```

```
<th>Photo</th>
```

```
<th>Name</th>
```

```
<th>Employee ID</th>
```

```
<th>JH/DMT</th>
```

```
<th>Phone</th>

<th>Date</th>

<th>System Rating</th>

<th>Improvement Category</th>

<th>User Comments</th>

</tr>

{% for emp in employees %}

<tr class="row">

    <td>

    </td>

    <td class="name">

        {{emp.name}}

    </td>

    <td class="empid">

        {{emp.employee_id}}

    </td>

    <td class="cat">

        {{emp.jh_name }}

    </td>
```



```
<td class="phone">
```

```
  {{emp.phone }}
```

```
</td>
```

```
<td>
```

```
  {{emp.submission_time }}
```

```
</td>
```

```
<td class="rating-cell">
```

```
  {% if emp.feedback_rating == 'Excellent' %}
```

```
    <span class="rating-badgerating-excellent">Excellent</span>
```

```
  {% elif emp.feedback_rating == 'Very Good' %}
```

```
    <span class="rating-badgerating-very-good">Very Good</span>
```

```
  {% elif emp.feedback_rating == 'Good' %}
```

```
    <span class="rating-badgerating-good">Good</span>
```

```
  {% elif emp.feedback_rating == 'Average' %}
```

```
    <span class="rating-badgerating-average">Average</span>
```

```
  {% elif emp.feedback_rating == 'Poor' %}
```

```
    <span class="rating-badgerating-poor">Poor</span>
```

```
  {% else %}
```

```
    <span class="rating-badger" style="background:#e2e8f0; color:
#475569;">N/A</span>
```

```
  {% endif %}
```

```
</td>
```

```
<td class="feedback-type">
```

```

        {{ emp.feedback_category if emp.feedback_category else 'None' }}
    </td>

    <td class="comment-text">

        {{emp.feedback_comments if emp.feedback_comments else 'No
comments left.'}}

    </td>

</tr>

{% endfor %}

</table>

```

```

<div class="no-data" id="noData">

```

No Records Found

```

</div>

```

```

<script>

```

```

function updateCounts(){

```

```

    let rows = document.querySelectorAll(".row");

```

```

    let total = rows.length;

```

```

    let visible = 0;

```

```

    let attentionRequired = 0;

```

```

    rows.forEach(row => {

```

```

        // Evaluate systemic warning metrics

```

```

        let ratingText = row.querySelector(".rating-cell").innerText.trim();

```

```

        if(ratingText === "Poor" || ratingText === "Average") {

```

```
        attentionRequired++;
    }

    if (row.style.display !== "none") {
        visible++;
    }
});
```

```
document.getElementById("totalCount").innerText = total;
document.getElementById("visibleCount").innerText = visible;
document.getElementById("attentionCount").innerText =
attentionRequired;
}
```

```
function filterTable() {
```

```
    let filter = document.getElementById("filter").value;
    let ratingFilter = document.getElementById("ratingFilter").value;
    let rows = document.querySelectorAll(". row");
    let hasData = false;
```

```
    rows.forEach(row => {
        let cat = row.querySelector(".cat").innerText;
        let rating = row.querySelector(". rating-cell").innerText.trim();
```

```
        let matchesCat = (filter === "ALL" || cat === filter);
```

```
let matchesRating = (ratingFilter === "ALL" || rating === ratingFilter);
if(matchesCat && matchesRating){
    row.style.display = "";
    hasData = true;
} else {
    row.style.display = "none";
}
});
```

```
document.getElementById("noData").style.display = hasData ? "none" :
"block";
updateCounts();
}
```

```
function searchTable(){
```

```
let input = document.getElementById("searchInput").value.toLowerCase();
let rows = document.querySelectorAll(". row");
let hasData = false;
```

```
rows.forEach(row => {
```

```
let name = row.querySelector(".name").innerText.toLowerCase();
let empid = row.querySelector(". empid").innerText.toLowerCase();
let phone = row.querySelector(". phone").innerText.toLowerCase();
```

```
// Let admin search elements inside feedback sections as well

let rating = row.querySelector(". rating-cell").innerText.toLowerCase();

let fbType = row.querySelector(". feedback-
type").innerText.toLowerCase();

let comments = row.querySelector(". comment-
text").innerText.toLowerCase();

if (
    name.includes(input) ||
    empid.includes(input) ||
    phone.includes(input) ||
    rating.includes(input) ||
    fbType.includes(input) ||
    comments.includes(input)
){
    row.style.display = "";
    hasData = true;
} else {
    row.style.display = "none";
}

});

document.getElementById("noData").style.display = hasData ? "none" :
"block";

updateCounts();
}
```

```
// INITIAL LOAD
```

```
updateCounts();
```

```
</script>
```

```
</body>
```

```
</html>
```

admin_login.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Admin Login</title>
```

```
  <style>
```

```
    body {background: #0f172a; color: white; font-family: sans-serif;
display: flex; justify-content: center; align-items: center; height: 100vh;
margin: 0;}
```

```
    .login-card {background: rgba (255,255,255,0.05); padding: 40px;
border-radius: 20px; text-align: center; width: 350px; border: 1px solid
rgba(255,255,255,0.1);}
```

```
    input {width: 100%; padding: 12px; margin: 20px 0; border-radius: 8px;
border: 1px solid #334155; background: #1e293b; color: white; font-size:
16px; box-sizing: border-box;}
```

```
    button {width: 100%; padding: 12px; background: #38bdf8; color:
#0f172a; border: none; border-radius: 8px; font-weight: bold; cursor: pointer;}
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <div class="login-card">
```

```
    <h2>Admin Authentication</h2>
```

```
    <form method="POST">
```

```
      <input type="password" name="password" placeholder="Enter System
Password" required>
```

```
      <button type="submit">UNLOCK DATABASE</button>
```

```
    </form>
```

```
  </div>
```

```
</body>
```

```
</html>
```

face_login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Face Login</title>
  <meta charset="UTF-8">
  <script
src="https://cdn.jsdelivr.net/npm/@mediapipe/camera_utils/camera_utils.js">
</script>
  <script
src="https://cdn.jsdelivr.net/npm/@mediapipe/control_utils/control_utils.js"><
/script>
  <script
src="https://cdn.jsdelivr.net/npm/@mediapipe/face_mesh/face_mesh.js"></scr
ipt>
  <style>
    body {
      background: #1e222d;
      color:white;
      font-family:sans-serif;
      padding:20px;
      display:flex;
      justify-content:center;
    }

    . card {
      background:rgba(255,255,255,0.05);
      max-width:650px;
      width:100%;
      padding:30px;
      border-radius:20px;
    }

    h2{
      text-align:center;
      margin-bottom:5px;
    }
```

```
p{
    text-align:center;
}

label {
    font-weight:bold;
    display:block;
    margin-top:15px;
    margin-bottom:8px;
}

.required {
    color:red;
}

input, select, textarea{
    width:100%;
    padding:12px;
    border-radius:10px;
    border:1px solid #555;
    background: #2d3748;
    color:white;
    box-sizing:border-box;
    margin-bottom:12px;
}

input: placeholder, textarea: placeholder {
    color: #ccc;
}

textarea{
    resize:vertical;
    font-family:sans-serif;
}

video {
    width:100%;
    border-radius:15px;
    border:2px solid #38bdf8;
    margin-top:15px;
    margin-bottom:20px;
```



```
    background:black;
}

.grid {
  display:grid;
  grid-template-columns:1fr 1fr;
  gap:15px;
  margin:20px 0;
}

.box {
  padding:15px;
  background: #4a5568;
  border-radius:10px;
  font-weight:bold;
  text-align:center;
  transition:0.3s;
}

.done {
  background: #2ecc71! important;
}

button {
  width:100%;
  padding:15px;
  background: #38bdf8;
  border:none;
  border-radius:10px;
  font-weight:bold;
  cursor:pointer;
  margin-top:10px;
}

button:disabled{
  background: #666;
  cursor:not-allowed;
}

#otpSection {
  display:none;
```

```
}

#otpStatus {
    font-weight:bold;
    margin-top:10px;
}

.policy-box {
    background: #2d3748;
    border-radius:10px;
    padding:15px;
    margin-top:15px;
    max-height:220px;
    overflow-y:auto;
}

.policy-box p {
    text-align:left;
    margin:10px 0;
    line-height:1.5;
}

#vBtn {
    display:none;
}

.error{
    color:#ff6b6b;
    font-size:13px;
    margin-top:-5px;
    margin-bottom:10px;
}

.feedback-section {
    background: rgba(56, 189, 248, 0.08);
    border: 1px dashed #38bdf8;
    border-radius: 15px;
    padding: 20px;
    margin-top: 25px;
    margin-bottom: 15px;
}
```

```

</style>
</head>
<body>

<div class="card">
  <h2>AUP & Password Policy</h2>
  <p>Digitalization System</p>

  <div style="background: #243447; padding:15px; border-radius:10px;
margin-bottom:20px; border-left:5px solid #38bdf8;">
    <h3 style="margin-top:0;">Current Active Policies</h3>
    <p><strong>Password Policy:</strong> Version 3.2</p>
    <p><strong>AUP Policy:</strong> <span
id="quarterText"></span></p>
    <p style="color: #ffcc00; font-weight:bold;">Employees must accept
latest policies before authentication. </p>
  </div>

  <div style="background: #fff3cd; padding:12px; border-radius:10px;
margin-bottom:20px; font-weight:bold; color:#856404; text-align:center;">
    ⚠ Next AUP & Password Policy revision scheduled for October 2026
  </div>

  <label>Authentication Mode <span class="required">*</span></label>
  <select id="auth_mode" onchange="onAuthModeChange()">
    <option value="live">Continue with Live Scan</option>
    <option value="registered">Continue with Registered Profile</option>
  </select>

  <label>Employee ID <span class="required">*</span></label>
  <input type="text" id="employee_id" maxlength="5" inputmode="numeric"
placeholder="Enter 5 Digit Employee ID">
  <div class="error" id="idError"></div>

  <label>Full Name <span class="required">*</span></label>
  <input type="text" id="name" placeholder="Enter Full Name">
  <div class="error" id="nameError"></div>

  <label>JH / DMT Category <span class="required">*</span></label>
  <select id="jh_dmt_select">
    <option value="">Select Category</option>

```

```
<optgroup label="JH CATEGORY">
  <option value="JH ISI">JH ISI</option>
  <option value="JH HR">JH HR</option>
  <option value="JH IT">JH IT</option>
  <option value="JH 1A">JH 1A</option>
  <option value="JH Department">JH Department</option>
  <option value="JH OTHERS">JH OTHERS</option>
</optgroup>
<optgroup label="DMT CATEGORY">
  <option value="DMT VETEND">DMT VETEND</option>
  <option value="DMT SEMISKILLED">DMT
SEMISKILLED</option>
  <option value="DMT TIME OFFICE">DMT TIME
OFFICE</option>
  <option value="DMT TRAINEE">DMT TRAINEE</option>
  <option value="DMT MANAGER">DMT MANAGER</option>
</optgroup>
</select>
```

```
<label>Phone Number <span class="required">*</span></label>
<input type="tel" id="phone" placeholder="Enter Phone Number">
<button type="button" onclick="sendOTP()">Send OTP</button>
```

```
<div id="otpSection">
  <input type="text" id="otp" placeholder="Enter OTP">
  <button type="button" onclick="verifyOTP()">Verify OTP</button>
</div>
<div id="otpStatus"></div>
```

```
<label>Email Address <span class="required">*</span></label>
<input type="email" id="email" placeholder="Enter Email Address">
<div class="error" id="emailError"></div>
```

```
<div class="policy-box">
  <h3>Company Protocols (Mandatory Acceptance) </h3>
  <p>1. I will ensure my own safety at workplace. </p>
  <p>2. I will follow all company rules and regulations. </p>
  <p>3. I will not share confidential information. </p>
  <p>4. I will use company systems responsibly. </p>
  <p>5. I will maintain discipline and punctuality. </p>
```

<p>6. I will respect colleagues and maintain teamwork. </p>
<p>7. I will not misuse company resources. </p>
<p>8. I will follow cybersecurity guidelines. </p>
<p>9. I will report suspicious activity immediately. </p>
<p>10. I accept all AUP and password security policies. </p>
</div>

<label>

<input type="checkbox" id="policy"> I agree to all mandatory protocols
and policies

</label>

<div id="liveSection">

<h2 style="margin-top:25px;">Liveness Check</h2>

<p id="msg">Move your face to complete the verification</p>

<video id="video" autoplay muted playsinline></video>

<div id="previewSection" style="display:none; text-align:center;">

<h3>Captured Photo Preview</h3>

<img id="capturedImage" style="width:220px; border-radius:10px;
border:3px solid #0074D9; box-shadow:0 4px 10px rgba(0,0,0,0.2);">

<div style="display:flex; justify-content:center; gap:10px;">

<button type="button" onclick="showPreviewPhoto()" style="background: #2196f3; width:220px;">Preview Photo</button>

<button type="button" onclick="recapturePhoto()" style="background: #ff9800; width:220px;">Recapture Photo</button>

</div>

<div id="largePreviewSection" style="display:none; text-align:center; margin-top:20px;">

</div>

</div>

</div>

<div id="registeredSection" style="display:none; margin-top:25px;">

<h2>Registered Profile</h2>

<p id="registeredMsg">Enter your Employee ID and Full Name, then

load your registered profile photo. </p>

<button type="button" onclick="loadRegisteredProfile()">Load
Registered Profile Photo</button>

<div id="registeredPhotoPreview" style="display:none; text-align:center;
margin-top:20px;">

<h3>Registered Profile Photo</h3>

<img id="registeredImage" style="width:220px; border-radius:10px;
border:3px solid #0074D9; box-shadow:0 4px 10px rgba(0,0,0,0.2);">

</div>

</div>

<div class="feedback-section">

<h3 style="margin-top:0; text-align:center; color: #38bdf8;">System
Feedback Loop</h3>

<label for="fb_rating">Overall Rating</label>

<select id="fb_rating">

<option value="Excellent">Excellent</option>

<option value="Very Good">Very Good</option>

<option value="Good">Good</option>

<option value="Average">Average</option>

<option value="Poor">Poor</option>

</select>

<label for="fb_suggestion_type">System Category
Improvements</label>

<select id="fb_suggestion_type">

<option value="No Suggestions">No Suggestions</option>

<option value="Improve User Interface">Improve User
Interface</option>

<option value="Faster Authentication Process">Faster Authentication
Process</option>

<option value="Improve Dashboard Features">Improve Dashboard
Features</option>

<option value="Add More Security Features">Add More Security
Features</option>

<option value="Improve Email Notifications">Improve Email
Notifications</option>

<option value="Other">Other</option>

</select>

```
<label for="fb_comments">Detailed User Feedback Comments</label>
<textarea id="fb_comments" rows="3" placeholder="Type your creative
suggestions or tracking portal experience notes here..."></textarea>
</div>
```

```
<button id="vBtn" onclick="sendVerify()">VERIFY IDENTITY</button>
</div>
```

```
<script>
const video = document.getElementById('video');
const vBtn = document.getElementById('vBtn');
const status = document.getElementById('msg');
const authModeSelect = document.getElementById('auth_mode');
const liveSection = document.getElementById('liveSection');
const registeredSection = document.getElementById('registeredSection');
const registeredPhotoPreview =
document.getElementById('registeredPhotoPreview');
const registeredImage = document.getElementById('registeredImage');
const registeredMsg = document.getElementById('registeredMsg');
```

```
let captured = false;
let faceStartTime = null;
let registeredLoaded = false;
let currentStream = null;
let generatedOTP = "";
let otpVerified = false;
let isProcessing = false;
```

```
/* =====
OTP
===== */
```

```
function sendOTP() {
  let phone = document.getElementById("phone").value;
  if (! /^[6-9]\d{9}$/.test(phone)){
    alert("Enter valid phone number");
    return;
  }
  generatedOTP = Math.floor(1000 + Math.random() * 9000).toString();
  alert("Demo OTP: " + generatedOTP);
}
```

```

    document.getElementById("otpSection").style.display = "block";
}

function verifyOTP(){
    let entered = document.getElementById("otp").value;
    if(entered === generatedOTP){
        otpVerified = true;
        document.getElementById("otpStatus").innerText = "OTP Verified ✓";
        document.getElementById("otpStatus").style.color = "#2ecc71";
    }else{
        document.getElementById("otpStatus").innerText = "Invalid OTP";
        document.getElementById("otpStatus").style.color = "red";
    }
}

function onAuthModeChange() {
    const mode = authModeSelect.value;
    if (mode === 'registered') {
        stopCamera();
        liveSection.style.display = 'none';
        registeredSection.style.display = 'block';
        registeredPhotoPreview.style.display = registeredLoaded ? 'block' :
'none';
        vBtn.style.display = registeredLoaded ? 'block' : 'none';
        status.innerText = 'Registered profile mode selected. Load your profile
photo.';
        status.style.color = '#ffffff';
    } else {
        registeredSection.style.display = 'none';
        registeredPhotoPreview.style.display = 'none';
        registeredLoaded = false;
        vBtn.style.display = "none";
        status.innerText = 'Move your face to complete the verification';
        status.style.color = 'white';
        recapturePhoto();
        startCamera();
    }
}

function startCamera() {
    if (currentStream) return;

```



```

navigator.mediaDevices.getUserMedia({ video: true })
  .then(stream => {
    currentStream = stream;
    video.srcObject = stream;
    video.onloadedmetadata = async () => {
      await video.play();
      function waitAndStart() {
        if (video.videoWidth === 0) {
          requestAnimationFrame(waitAndStart);
          return;
        }
        const loop = async () => {
          if (video.readyState === 4 && !captured && !isProcessing &&
authModeSelect.value === 'live') {
            isProcessing = true;
            await faceMesh.send({ image: video });
            isProcessing = false;
          }
          requestAnimationFrame(loop);
        };
        loop();
      }
      waitAndStart();
    };
  })
  .catch(err => {
    console.error(err);
    status.innerText = "Camera blocked or permission denied";
    status.style.color = "red";
  });
}

function stopCamera() {
  if (currentStream) {
    currentStream.getTracks().forEach(track => track.stop());
    currentStream = null;
    video.srcObject = null;
  }
}

async function loadRegisteredProfile() {

```

```
const employeeId = document.getElementById('employee_id').value.trim();
const name = document.getElementById('name').value.trim();

if (!/^\d{5}$/.test(employeeId)) {
    alert('Employee ID must be exactly 5 digits');
    return;
}

if (name.length < 3) {
    alert('Enter valid name');
    return;
}

const res = await fetch('/load_registered_photo', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ employee_id: employeeId, name })
});

const data = await res.json();

if (data.status === 'success') {
    registeredLoaded = true;
    registeredPhotoPreview.style.display = 'block';
    registeredImage.src = data.photo_data;
    vBtn.style.display = 'block';
    registeredMsg.innerText = 'Registered profile photo loaded successfully.';
    status.innerText = 'Registered profile loaded. Click VERIFY  
IDENTITY.';
    status.style.color = '#2ecc71';
} else {
    registeredLoaded = false;
    registeredPhotoPreview.style.display = 'none';
    vBtn.style.display = 'none';
    alert(data.message);
}
}
```

```

/* =====
RECAPTURE PHOTO
===== */
function recapturePhoto(){
    captured = false;
    faceStartTime = null;
    document.getElementById("previewSection").style.display = "none";
    document.getElementById("largePreviewSection").style.display = "none";
    document.getElementById("capturedImage").src = "";
    status.innerText = "Center your face and look straight";
    status.style.color = "white";
    vBtn.style.display = "none";
}

/* =====
SHOW PHOTO IN SEPARATE NEW WINDOW
===== */
function showPreviewPhoto(){
    const imgData = document.getElementById("capturedImage").src;
    if(!imgData){
        alert("No photo captured");
        return;
    }

    const previewWindow = window.open("", "_blank",
    "width=600,height=500,scrollbars=yes");
    if(previewWindow) {
        previewWindow.document.write(`
        <html>
        <head>
        <title>Image Preview</title>
        <style>
        body {
            background: #1e222d;
            color: white;
            font-family: sans-serif;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            margin: 0;

```

```

        padding: 20px;
        height: 100vh;
        box-sizing: border-box;
    }
    h3 {
        margin-bottom: 20px;
    }
    img {
        max-width: 100%;
        max-height: 80vh;
        border-radius: 10px;
        border: 3px solid #38bdf8;
        box-shadow: 0 4px 15px rgba(0,0,0,0.5);
    }
</style>
</head>
<body>
    <h3>Image Preview</h3>
    
</body>
</html>
`);
previewWindow.document.close();
} else {
    alert("Pop-up blocked! Please allow popups for this digital system to
view previews.");
}
}

/* =====
FACE MESH
===== */
const faceMesh = new FaceMesh({
    locateFile: (file) =>
`https://cdn.jsdelivr.net/npm/@mediapipe/face_mesh/${file}`
});

faceMesh.setOptions({
    maxNumFaces: 1,
    refineLandmarks: true
});

```

```
faceMesh.onResults(results => {
  if(captured) return;

  if(results.multiFaceLandmarks && results.multiFaceLandmarks.length >
0){
    if(results.multiFaceLandmarks.length > 1){
      faceStartTime = null;
      status.innerText = "⚠ 2 or more people detected. Only 1 person is
allowed.";
      status.style.color = "red";
      return;
    }

    const lm = results.multiFaceLandmarks[0];
    const nose = lm[1];

    if(nose.x > 0.35 && nose.x < 0.65 && nose.y > 0.30 && nose.y < 0.70){
      status.innerText = "Keep looking straight...";
      status.style.color = "#38bdf8";

      if(!faceStartTime){
        faceStartTime = Date.now();
      }

      if(Date.now() - faceStartTime >= 2000){
        captured = true;
        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);

        const capturedData = canvas.toDataURL('image/jpeg');
        document.getElementById("capturedImage").src = capturedData;
        document.getElementById("previewSection").style.display =
"block";

        status.innerText = "Photo Captured ✓";
        status.style.color = "#2ecc71";
        vBtn.style.display = "block";
      }
    }
  }
});
```

```

        }else{
            faceStartTime = null;
            status.innerText = "Center your face and look straight";
            status.style.color = "#ff6b6b";
        }
    }else{
        faceStartTime = null;
        status.innerText = "No face detected";
        status.style.color = "#ff6b6b";
    }
});

/* =====
    VERIFY
===== */
async function sendVerify() {
    const empId = document.getElementById("employee_id").value.trim();
    if(!/^\d{5}$/.test(empId)){
        alert("Employee ID must be exactly 5 digits");
        return;
    }

    const name = document.getElementById("name").value.trim();
    if(name.length < 3){
        alert("Enter valid name");
        return;
    }

    const jh = document.getElementById("jh_dmt_select").value;
    if(jh === ""){
        alert("Select JH/DMT category");
        return;
    }

    const phone = document.getElementById("phone").value.trim();
    if(!/^[6-9]\d{9}$/.test(phone)){
        alert("Invalid phone number");
        return;
    }
}

```

```
if(!otpVerified){  
    alert("Verify OTP first");  
    return;  
}
```

```
const email = document.getElementById("email").value.trim();  
if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)){  
    alert("Invalid email");  
    return;  
}
```

```
if(!document.getElementById("policy").checked){  
    alert("Accept all policies");  
    return;  
}
```

```
const mode = authModeSelect.value;
```

```
// Capture feedback values safely to send to backend server logic  
const feedbackRating = document.getElementById("fb_rating").value;  
const feedbackCategory =  
document.getElementById("fb_suggestion_type").value;  
const feedbackComments =  
document.getElementById("fb_comments").value.trim();
```

```
let payload = {  
    mode,  
    employee_id: empId,  
    name,  
    feedback: {  
        rating: feedbackRating,  
        category: feedbackCategory,  
        comments: feedbackComments  
    }  
};
```

```
if (mode === 'live') {  
    if (!captured) {  
        alert('Please complete the live scan and capture your face.');        return;  
    }  
}
```

```

    payload.photo_data = document.getElementById("capturedImage").src;
}

const res = await fetch('/verify', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(payload)
});

const data = await res.json();
alert(data.message);

if(data.status === 'success'){
  window.location.href = "/";
}
}

/* =====
CAMERA RUN ON INITIAL LOAD
===== */
onAuthModeChange();

/* =====
AUTO QUARTER SYSTEM
===== */
const now = new Date();
const month = now.getMonth() + 1;
const year = now.getFullYear();
let quarter = "";

if(month >= 1 && month <= 3){
  quarter = "Quarter 1";
}else if(month >= 4 && month <= 6){
  quarter = "Quarter 2";
}else if(month >= 7 && month <= 9){
  quarter = "Quarter 3";
}else{
  quarter = "Quarter 4";
}

```



```
document.getElementById("quarterText").innerText = quarter + " - " + year;

window.onload = () => {
    startCamera();
};
</script>
</body>
</html>
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>AUP Registration Form</title>
</head>

<body>

<h2>Employee Registration & AUP Form</h2>

<form action="/submit" method="POST">

    PSWD ID:<br>
    <input type="text" name="pswd_id" required>
    <br><br>

    Employee Name:<br>
    <input type="text" name="name" required>
    <br><br>

    JH Name:<br>
    <input type="text" name="jh_name" required>
    <br><br>

    DMT Name:<br>
    <input type="text" name="dmt_name" required>
    <br><br>
```

Phone Number:

<input type="text" name="phone" required>

Email ID:

<input type="email" name="email" required>

Department:

<input type="text" name=" department" required>

<h3>Company Safety & Usage Policies</h3>

<input type="checkbox" required>

I will ensure my own safety.

<input type="checkbox" required>

I will follow company protocols and rules.

<input type="checkbox" required>

I will wear safety shoes and helmet in required areas.

<input type="checkbox" required>

I will maintain workplace discipline.

<input type="checkbox" required>

I will not misuse company systems or resources.

<input type="checkbox" required>

I will follow data privacy and security guidelines.

<input type="checkbox" required>

I will report unsafe conditions immediately.

```
<br><br>
```

```
<input type="" checkbox" required>  
I will use company assets responsibly.  
<br><br>
```

```
<input type="" checkbox" required>  
I will follow attendance and work timing policies.  
<br><br>
```

```
<input type="" checkbox" required>  
I confirm that all provided details are correct.  
<br><br>
```

```
<a href="/camera">  
  <button type="" button">Capture Live Photo</button>  
</a>
```

```
<br><br>
```

```
<button type="" submit">Register</button>
```

```
</form>
```

```
</body>  
</html>
```

main.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>DIGITALIZATION OF AUP</title>  
  
  <style>  
    *{  
      margin:0;
```

```
padding:0;
box-sizing:border-box;
font-family: Arial, sans-serif;
}

body {
  height:100vh;
  background: linear-gradient(135deg,#001f3f,#0074D9,#7FDBFF);
  display:flex;
  justify-content:center;
  align-items:center;
  overflow:hidden;
}

.main-container {
  text-align:center;
  color:white;
  animation: fadeIn 2s ease;
}

/* Updated these two classes to be identical */
.main-title, sub-title {
  font-size: 70px;
  font-weight: bold;
  text-shadow: 3px 3px 10px rgba(0,0,0,0.5);
  text-transform: uppercase;
  margin-bottom: 10px;
}

.sub-title {
  margin-bottom: 50px; /* Space before the button */
}

.btn{
  padding:15px 45px;
```

```
font-size:22px;
border:none;
border-radius:50px;
background:white;
color:#003366;
cursor:pointer;
transition:0.4s;
font-weight:bold;
text-decoration:none;
display: inline-block;
box-shadow: 0 4px 15px rgba(0,0,0,0.2);
}
```

```
.btn:hover{
  background: #003366;
  color:white;
  transform:scale(1.08);
  box-shadow:0px 0px 20px white;
}
```

```
@keyframes fadeIn{
  from {
    opacity:0;
    transform:translateY(-30px);
  }
  to {
    opacity:1;
    transform:translateY(0px);
  }
}
</style>
```

```
</head>
```

```
<body>
```

```
<div class="main-container">
```

```
  <div class="main-title">
```

```
    DIGITALIZATION OF AUP
```

```
  </div>
```

```
  <div class="sub-title">
```

```
    & PASSWORD POLICY
```

```
  </div>
```

```
  <a href="{{ url_for('selection') }}" class="btn">
```

```
    ENTER SYSTEM
```

```
  </a>
```

```
</div>
```

```
</body>
```

```
</html>
```

password.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Create Password</title>
```

```
  <style>
```

```
body{
```

```
  font-family: Arial;
```

```
  background-color: #f2f2f2;
```

```
  display:flex;
```

```
    justify-content:center;
    align-items:center;
    height:100vh;
}
```

```
.container{
    background:white;
    padding:30px;
    border-radius:10px;
    width:350px;
    box-shadow:0px 0px 10px rgba(0,0,0,0.2);
}
```

```
h2{
    text-align:center;
}
```

```
input{
    width:100%;
    padding:10px;
    margin-top:8px;
    margin-bottom:15px;
}
```

```
button{
    width:100%;
    padding:10px;
    background-color:blue;
    color:white;
    border:none;
    cursor:pointer;
}
```

```
button:hover{
    background-color:darkblue;
```

```
}
```

```
.rules{  
    font-size:14px;  
    color:gray;  
}
```

```
.error{  
    color:red;  
    text-align:center;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2>Create Strong Password</h2>
```

```
{% if message %}  
    <p class="error">{{ message }}</p>  
{% endif %}
```

```
<form method="POST">
```

```
<label>Enter Password</label>
```

```
<input type="password" name="password" required>
```

```
<label>Confirm Password</label>
```

```
<input type="password" name="confirm_password" required>
```

```
<div class="rules">
```

```
    Password must contain:
```



```
<ul>
  <li>Minimum 8 characters</li>
  <li>One uppercase letter</li>
  <li>One lowercase letter</li>
  <li>One number</li>
  <li>One special character</li>
</ul>
</div>

<button type="submit">Submit</button>

</form>

</div>

</body>
</html>
```

records.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Records</title>
</head>
<body>

<h2>📋 Submitted Employee Records</h2>

<table border="1" cellpadding="10">
  <tr>
    <th>ID</th>
    <th>Name</th>
    <th>JH Name</th>
```

```
<th>DMT Name</th>
<th>Phone</th>
<th>Email</th>
<th>Photo</th>
<th>Time</th>
</tr>
```

```
{% for emp in employees %}
```

```
<tr>
```

```
<td>{{ emp[0] }}</td>
```

```
<td>{{ emp[1] }}</td>
```

```
<td>{{ emp[2] }}</td>
```

```
<td>{{ emp[3] }}</td>
```

```
<td>{{ emp[4] }}</td>
```

```
<td>{{ emp[5] }}</td>
```

```
<td>
```

```

```

```
</td>
```

```
<td>{{ emp[7] }}</td>
```

```
</tr>
```

```
{% endfor %}
```

```
</table>
```

```
<br>
```

```
<a href="/">← Back to Form</a>
```

```
</body>
```

```
</html>
```

selection.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
  <style>
    body { background: #0f172a; height: 100vh; display: flex; justify-
content: center; align-items: center; margin: 0; font-family: sans-serif; }
    .wrapper { display: flex; gap: 40px; }
    .card {
      background: rgba(255, 255, 255, 0.05);
      border: 1px solid rgba(255, 255, 255, 0.1);
      width: 300px; padding: 60px 20px; border-radius: 30px;
      text-align: center; text-decoration: none; color: white;
      transition: 0.4s; backdrop-filter: blur(10px);
    }

    .card:hover { transform: translateY(-20px); border-color: #38bdf8;
background: rgba(255, 255, 255, 0.1); }
    .icon-circle {
      width: 100px; height: 100px; background: rgba(255,255,255,0.1);
      border-radius: 50%; display: flex; align-items: center; justify-content:
center;
      margin: 0 auto 30px; font-size: 40px; color: #38bdf8;
    }
  </style>
</head>
<body>
  <div class="wrapper">
    <a href="{{ url_for('admin_login') }}" class="card">
      <div class="icon-circle"><i class="fa-solid fa-user-shield"></i></div>
      <h2>ADMIN</h2>
    </a>
  </div>
</body>
</html>
```

```
<p>System Management</p>
</a>
<a href="{{ url_for('user_options') }}" class="card">
  <div class="icon-circle"><i class="fa-solid fa-users"></i></div>
  <h2>USER</h2>
  <p>Employee Portal</p>
</a>
</div>
</body>
</html>
```

success.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Success</title>

  <style>
    body{
      margin:0;
      padding:0;
      font-family:Arial, sans-serif;
      height:100vh;
      display:flex;
      justify-content:center;
      align-items:center;
      background:linear-gradient(135deg,#11998e,#38ef7d);
    }

    /* GLASS CARD WITH ANIMATION */
    .box{
      background: rgba(255, 255, 255, 0.92);
```

```
    backdrop-filter: blur(12px);
    padding:50px;
    border-radius:25px;
    text-align:center;
    box-shadow:0px 20px 40px rgba(0,0,0,0.2);
    width: 380px;
    animation: fadeIn 0.6s ease;
}
```

```
@keyframes fadeIn{
    from{opacity:0; transform:translateY(20px);}
    to{opacity:1; transform:translateY(0);}
}
```

```
/* TICK WITH POP ANIMATION */
.tick{
    font-size:70px;
    margin-bottom:20px;
    animation: pop 0.5s ease;
}
```

```
@keyframes pop{
    0%{transform:scale(0);}
    100%{transform:scale(1);}
}
```

```
h1{
    color:#0f766e;
    margin-bottom:20px;
}
```

```
p{
    font-size:20px;
    color:#555;
```

```
}
```

```
/* ===== */
```

```
/* EMAIL VERIFICATION BADGE */
```

```
/* ===== */
```

```
. badge {  
    width: 90px;  
    height: 90px;  
    background: #22c55e;  
    border-radius: 50%;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    margin: 0 auto 20px;  
    color: white;  
    font-size: 40px;  
    box-shadow: 0 0 15px rgba(0,0,0,0.2);  
}
```

```
. verified-text {  
    margin-top: 10px;  
    font-size: 18px;  
    color: #16a34a;  
    font-weight: bold;  
}
```

```
/* ===== */
```

```
/* FACE MATCH SECTION */
```

```
/* ===== */
```

```
.match-box {  
    margin-top: 25px;
```

```
padding: 15px;
border-radius: 12px;
background: #ecfdf5;
border: 1px solid #86efac;
}
```

```
.match-title {
  font-size: 16px;
  color: #14532d;
  margin-bottom: 8px;
  font-weight: bold;
}
```

```
.match-value {
  font-size: 28px;
  color: #16a34a;
  font-weight: bold;
}
```

```
.bar {
  height: 10px;
  width: 100%;
  background: #d1fae5;
  border-radius: 20px;
  margin-top: 10px;
  overflow: hidden;
}
```

```
.bar-fill {
  height: 100%;
  width: 96%;
  background: #22c55e;
  border-radius: 20px;
}
```

</style>

</head>

<body>

<div class="box">

<div class="tick"> </div>

<h1>Welcome Aboard</h1>

<p>Registration Completed Successfully</p>

<div class="badge">✓</div>

<div class="verified-text">
Email Verified Successfully
</div>

<div class="match-box">

<div class="match-title">
Face Match Result
</div>

<div class="match-value">
96%
</div>

<div class="bar">
<div class="bar-fill"></div>
</div>

</div>

</div>

</body>

</html>

templates.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Records</title>
</head>
<body>

<h2>📋 Submitted Employee Records</h2>

<table border="1" cellpadding="10">
  <tr>
    <th>ID</th>
    <th>Name</th>
    <th>JH Name</th>
    <th>DMT Name</th>
    <th>Phone</th>
    <th>Email</th>
    <th>Photo</th>
    <th>Time</th>
  </tr>

  {% for emp in employees %}
  <tr>
    <td> {{ emp [0] }} </td>
    <td> {{ emp [1] }} </td>
    <td> {{ emp [2] }} </td>
    <td> {{ emp [3] }} </td>
    <td> {{ emp [4] }} </td>
    <td> {{ emp [5] }} </td>
    <td>
      
    </td>
  </tr>
  {% endfor %}
</table>
```

```

        </td>
        <td> {{emp [7]}} </td>
    </tr>
    {% endfor %}
</table>

<br>
<a href="/">← Back to Form</a>

</body>
</html>

```

user_options.html

```

<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <style>
        body {background: #0f172a; height: 100vh; display: flex; justify-content:
center; align-items: center; gap: 30px; font-family: sans-serif; }
        .btn-card {
            background: white; padding: 40px; border-radius: 20px; width: 280px;
            text-align: center; text-decoration: none; color: #0f172a; font-weight:
bold;
            transition: 0.3s; box-shadow: 0 10px 30px rgba(0,0,0,0.5);
        }
        .btn-card:hover { transform: scale(1.05); background: #f1f5f9; }
        .icon { font-size: 50px; color: #0074D9; margin-bottom: 15px; }
    </style>
</head>
<body>

    <a href="{{ url_for('register') }}" class="btn-card">

```

```
<div class="icon"><i class="fa-solid fa-user-plus"></i></div>
```

New Registration

```
</a>
```

```
<a href="{{ url_for('face_login') }}" class="btn-card">
```

```
<div class="icon"><i class="fa-solid fa-camera-retro"></i></div>
```

Already Registered?

```
</a>
```

```
</body>
```

```
</html>
```

view.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>AUP AND Password Policy</title>
```

```
<script
```

```
src="https://cdn.jsdelivr.net/npm/@mediapipe/face_mesh"></script>
```

```
<style>
```

```
body{
```

```
font-family:'Segoe UI',sans-serif;
```

```
background:linear-gradient(135deg,#e0f7fa,#f1f8e9);
```

```
padding:40px;
```

```
display:flex;
```

```
justify-content:center;
```

```
}
```

```
.form-box{
```

```
background:rgba(255,255,255,0.95);
backdrop-filter:blur(10px);
padding:30px;
border-radius:18px;
width:520px;
box-shadow:0 15px 35px rgba(0,0,0,0.15);
}
```

```
h2{
  color:#0f766e;
}
```

```
p{
  color:#666;
}
```

```
label{
  font-weight:bold;
}
```

```
.required{
  color:red;
  font-size:18px;
}
```

```
input,select{
  width:100%;
  padding:12px;
  margin:8px 0 15px 0;
  border:1px solid #ddd;
  border-radius:8px;
  box-sizing:border-box;
}
```

```
.error{
```

```
    color:red;
    font-size:13px;
    margin-top:-10px;
    margin-bottom:10px;
}
```

```
video{
    width:100%;
    border-radius:10px;
    background:#000;
    margin-top:10px;
}
```

```
button{
    width:100%;
    padding:15px;
    background:linear-gradient(135deg,#0074D9,#00bcd4);
    color:white;
    border:none;
    border-radius:10px;
    font-weight:bold;
    cursor:pointer;
    transition:0.3s;
}
```

```
button:hover{
    transform:scale(1.02);
}
```

```
button:disabled{
    background:#ccc;
    transform:none;
    cursor:not-allowed;
}
```

```
#status{  
    font-weight:bold;  
    margin:10px 0;  
    color:#d93025;  
}
```

```
.otp-btn{  
    margin-top:-5px;  
    margin-bottom:15px;  
}
```

```
#otpSection{  
    display:none;  
}
```

```
#timer{  
    font-size:14px;  
    color:#0074D9;  
    margin-bottom:10px;  
}
```

```
#otpStatus{  
    font-weight:bold;  
    margin-bottom:10px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="form-box">
```

```
<div style="text-align:center; margin-bottom:15px;">
```

```

```

```
</div>
```

```
<h2 style="text-align:center; margin-bottom:5px;">
  AUP & Password Policy
</h2>
```

```
<p style="text-align:center; margin-top:0;">
  Digitalization System
</p>
```

```
<div style="
background:#24364d;
border-left:5px solid #38bdf8;
border-radius:15px;
padding:25px;
margin-top:20px;
margin-bottom:25px;
color:white;
">
```

```
<h2 style="
  margin-top:0;
  color:white;
  font-size:20px;
```

">

Current Active Policies

</h2>

<div style="

text-align:center;

line-height:2;

font-size:18px;

font-weight:bold;

">

<div>

Password Policy:

Version 3.2

</div>

<div>

AUP Policy:

</div>

</div>

<p style="

text-align:center;

color:#ffd60a;

font-size:18px;

font-weight:bold;

margin-top:20px;

">

Employees must accept latest policies before authentication.

</p>

</div>

```
<div style="
  background:#efe2b9;
  color:#8a6500;
  padding:18px;
  border-radius:15px;
  text-align:center;
  font-size:16px;
  font-weight:bold;
  margin-bottom:25px;
">
```

⚠ Next AUP & Password Policy revision scheduled for October 2026

</div>

```
<form action="/submit"
  method="POST"
  onsubmit="return validateForm()">
```

<label>

Employee ID *

</label>

<input

type="text"

name="employee_id"

id="employee_id"

required>

```
<div class="error" id="idError"></div>
```

```
<label>
```

```
Full Name <span class="required">*</span>
```

```
</label>
```

```
<input
```

```
type="text"
```

```
name="name"
```

```
id="name"
```

```
required>
```

```
<div class="error" id="nameError"></div>
```

```
<label>
```

```
JH / DMT Category <span class="required">*</span>
```

```
</label>
```

```
<select
```

```
name="jh_dmt_name"
```

```
id="jh_dmt_select"
```

```
onchange="toggleOther(this)"
```

```
required>
```

```
<option value="">Select Category</option>
```

```
<optgroup label="JH CATEGORY">
```

```
<option value="JH ISI">JH ISI</option>
```

```
<option value="JH HR">JH HR</option>
```

```
<option value="JH IT">JH IT</option>
```

```
<option value="JH 1A">JH 1A</option>
```

```
<option value="JH Department">JH Department</option>
```

```
<option value="JH OTHERS">JH OTHERS</option>
```

</optgroup>

<optgroup label="DMT CATEGORY">

<option value="DMT VETEND">DMT VETEND</option>

<option value="DMT SEMISKILLED">DMT
SEMISKILLED</option>

<option value="DMT TIME OFFICE">DMT TIME
OFFICE</option>

<option value="DMT TRAINEE">DMT TRAINEE</option>

<option value="DMT MANAGER">DMT MANAGER</option>

</optgroup>

<option value="OTHER">

OTHER (Type manually)

</option>

</select>

<input

type="text"

name="jh_dmt_manual"

id="jh_dmt_manual"

placeholder="Enter custom JH/DMT"

style="display:none;">

<div class="error" id="jhDmtError"></div>

<label>

Phone Number *

</label>

<input

```
type="tel"
name="phone"
id="phone"
required>
```

```
<button
  type="button"
  class="otp-btn"
  onclick="sendOTP()">
```

Send OTP

```
</button>
```

```
<div id="otpSection">
```

```
<input
  type="text"
  id="otp"
  placeholder="Enter 4 Digit OTP">
```

```
<div id="timer"></div>
```

```
<button
  type="button"
  onclick="verifyOTP()"
  style="margin-bottom:15px;">
```

Verify OTP

```
</button>
```

```
</div>
```

```
<div id="otpStatus"></div>
```

```
<div class="error" id="phoneError"></div>
```

```
<label>
```

```
  Email Address <span class="required">*</span>
```

```
</label>
```

```
<input
```

```
  type="email"
```

```
  name="email"
```

```
  id="email"
```

```
  required>
```

```
<div class="error" id="emailError"></div>
```

```
<p id="status">
```

```
  Status: Detecting Face...
```

```
</p>
```

```
<video id="video" autoplay muted></video>
```

```
<input
```

```
  type="hidden"
```

```
  name="photo_data"
```

```
  id="photo_data">
```

```
<div id="previewSection"
```

```
  style="
```

```
  display:none;
```

```
  text-align:center;
```

```
  margin-top:20px;
```

```
">
```

```
<h3 style="color:#0074D9;">
```

```
  Captured Photo Preview
```

</h3>

```
<img
  id="capturedImage"
  style="
    width:220px;
    border-radius:10px;
    border:3px solid #0074D9;
    box-shadow:0 4px 10px rgba(0,0,0,0.2);
  ">
<br><br>
```

```
<button
  type="button"
  onclick="showPreviewPhoto()"
  style="
    background: #0074D9;
    width:220px;
  ">
```

Preview Photo

```
</button>
```

```
<br><br>
<br><br>
```

```
<button
  type="button"
  onclick="recapturePhoto()"
  style="
    background: #ff9800;
    width:220px;
  ">
```

Recapture Photo

</button>

</div>

<div style="background: #f8fafc; padding:20px; border-radius:12px; border:1px solid #e2e8f0; margin-bottom:20px;">

 <h3 style="margin-top:0; color:#0f766e; font-size:16px; margin-bottom:15px;">System & Registration Feedback</h3>

 <label>Rate Your Experience *</label>

 <select name="feedback_rating" required>

 <option value="">Choose Rating</option>

 <option value="Excellent">Excellent</option>

 <option value="Very Good">Very Good</option>

 <option value="Good">Good</option>

 <option value="Average">Average</option>

 <option value="Poor">Poor</option>

 </select>

 <label>Area of Improvement</label>

 <select name="feedback_category">

 <option value="None">None (Perfect)</option>

 <option value="Face Verification Speed">Face Verification Speed</option>

 <option value="Camera Connectivity">Camera Connectivity</option>

 <option value="OTP Delivery Lag">OTP Delivery Lag</option>

 <option value="Interface Design">Interface Design</option>

 </select>

<label>Comments / Issues Faced</label>

<input type="text" name="feedback_comments"
placeholder="Describe any problems or suggestions..." style="margin-
bottom:5px;">

</div>

<div style="margin-top:15px;">

<label style="font-weight:bold;">

Mandatory Security Policies

*

</label>

<div style="
border:1px solid #ccc;
padding:15px;
border-radius:10px;
background:#f9f9f9;
margin-top:10px;
max-height:220px;
overflow-y:auto;
">

<p>

1. Employee must maintain confidentiality of company data.

</p>

<p>

2. Sharing passwords with others is strictly prohibited.

</p>

<p>

3. Unauthorized system access is not permitted.

</p>

<p>

4. Face authentication is mandatory for verification.

</p>

<p>

5. Employees must follow AUP security standards.

</p>

<p>

6. Fake registrations will lead to access denial.

</p>

<p>

7. Official email ID must be used during registration.

</p>

<p>

8. Mobile number verification is compulsory.

</p>

<p>

9. Employees are responsible for account activities.

</p>

<p>

10. Use helmet, gloves, apron and shoes in working area.

</p>

</div>

<label>

```
<input
  type="checkbox"
  id="policy"
  required>
```

I agree to all mandatory protocols and security policies

```
</label>
```

```
</div>
```

```
<br><br>
```

```
<button
  type="submit"
  id="submitBtn"
  disabled>
```

REGISTER NOW

```
</button>
```

```
</form>
```

```
</div>
```

```
<script>
```

```
let otpVerified = false;
let generatedOTP = "";
let timerInterval;
let captured = false;
let faceStartTime = null;
```

```
/* =====  
FORM VALIDATION  
===== */
```

```
function validateForm(){  
  
    let valid = true;  
  
    const empId =  
    document.getElementById("employee_id").value.trim();  
  
    const name =  
    document.getElementById("name").value.trim();  
  
    const select =  
    document.getElementById("jh_dmt_select").value;  
  
    const manual =  
    document.getElementById("jh_dmt_manual").value.trim();  
  
    const phone =  
    document.getElementById("phone").value.trim();  
  
    const email =  
    document.getElementById("email").value.trim();  
  
    document.getElementById("idError").innerText = "";  
    document.getElementById("nameError").innerText = "";  
    document.getElementById("jhDmtError").innerText = "";  
    document.getElementById("phoneError").innerText = "";  
    document.getElementById("emailError").innerText = "";  
  
    if(empId.length < 3){
```

```
document.getElementById("idError").innerText =
"Invalid Employee ID";

valid = false;
}

if(name.length < 3){

document.getElementById("nameError").innerText =
"Invalid Name";

valid = false;
}

if(select === "" ||
(select === "OTHER" && manual.length < 2)){

document.getElementById("jhDmtError").innerText =
"Select valid JH/DMT";

valid = false;
}

if(!/^[6-9]\d{9}$/.test(phone)){

document.getElementById("phoneError").innerText =
"Invalid Phone";

valid = false;
}

if(!otpVerified){

alert("Please verify OTP first");
```

```
        valid = false;
    }

    if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)){

        document.getElementById("emailError").innerText =
            "Invalid Email";

        valid = false;
    }

    return valid;
}

/* =====
   JH DMT
   ===== */

function toggleOther(select){

    let manual =
        document.getElementById("jh_dmt_manual");

    if(select.value === "OTHER"){

        manual.style.display = "block";
        manual.required = true;

    }else{

        manual.style.display = "none";
        manual.required = false;
        manual.value = "";
    }
}
```

```
/* =====  
OTP  
===== */
```

```
function sendOTP(){
```

```
    let phone =  
    document.getElementById("phone").value;
```

```
    if(! /^[6-9]\d{9}$/.test(phone)){
```

```
        alert("Enter valid phone number");  
        return;  
    }
```

```
    generatedOTP =  
    Math.floor(1000 + Math.random() * 9000).toString();
```

```
    alert("Demo OTP: " + generatedOTP);
```

```
    document.getElementById("otpSection").style.display =  
    "block";
```

```
    let timeLeft = 60;
```

```
    clearInterval(timerInterval);
```

```
    timerInterval = setInterval(() => {
```

```
        timeLeft--;
```

```
        document.getElementById("timer").innerText =  
        "OTP expires in " + timeLeft + " seconds";
```

```
    if(timeLeft <= 0){

        clearInterval(timerInterval);

        generatedOTP = "";

        document.getElementById("timer").innerText =
            "OTP Expired";
    }

    },1000);
}

function verifyOTP(){

    let enteredOTP =
    document.getElementById("otp").value;

    if(enteredOTP === generatedOTP &&
        generatedOTP !== ""){

        otpVerified = true;

        document.getElementById("otpStatus").innerText =
            "OTP Verified ✓";

        document.getElementById("otpStatus").style.color =
            "green";

        checkSubmitStatus();

    }else{

        document.getElementById("otpStatus").innerText =
```

```
"Invalid OTP";

document.getElementById("otpStatus").style.color =
"red";
}
}

/* =====
RECAPTURE
===== */

function recapturePhoto(){

    captured = false;

    document.getElementById("previewSection").style.display =
"none";

    document.getElementById("capturedImage").src = "";

    status.innerText =
"Move your face to complete the verification";

    status.style.color =
"white";
}

function showPreviewPhoto(){

    let imageSrc =
document.getElementById("capturedImage").src;
    console.log(imageSrc);

    if(!imageSrc){
        alert("No image captured yet");
    }
}
```



```
    return;
}

let previewWindow = window.open(
    "",
    "PhotoPreview",
    "width=700,height=700"
);

previewWindow.document.write(`
    <html>
    <head>
        <title>Captured Photo</title>
    </head>
    <body style="
        margin:0;
        display:flex;
        justify-content:center;
        align-items:center;
        background:#000;
        height:100vh;
    ">
        
        </body>
    </html>
`);

previewWindow.document.close();
}
```

```
/* =====  
SUBMIT STATUS  
===== */  
  
const video =  
document.getElementById('video');  
  
const status =  
document.getElementById('status');  
  
const photoInput =  
document.getElementById('photo_data');  
  
const submitBtn =  
document.getElementById('submitBtn');  
  
const policy =  
document.getElementById('policy');  
  
function checkSubmitStatus(){  
  
    if(captured &&  
        otpVerified &&  
        policy.checked){  
  
        submitBtn.disabled = false;  
  
    }else{  
  
        submitBtn.disabled = true;  
    }  
}  
  
policy.addEventListener(  
    'change',
```

```
        checkSubmitStatus
    );

    /* =====
    FACEMESH
    ===== */

    const faceMesh = new FaceMesh({

        locateFile: (file) =>
            `https://cdn.jsdelivr.net/npm/@mediapipe/face_mesh/${file}`
    });

    faceMesh.setOptions({

        maxNumFaces:5,
        refineLandmarks:true
    });

    faceMesh.onResults(results => {

        if(captured) return;

        if(results.multiFaceLandmarks &&
            results.multiFaceLandmarks.length > 0){
            if(results.multiFaceLandmarks.length > 1){

                faceStartTime = null;

                status.innerText =
                    "⚠ 2 or more people detected. Only 1 person is allowed.";

                status.style.color =
```

```
"red";

return;
}

    const lm =
    results.multiFaceLandmarks[0];

    const nose = lm[1];
if(
    nose.x > 0.40 &&
    nose.x < 0.60 &&
    nose.y > 0.30 &&
    nose.y < 0.70
){
    status.innerText =
    "Keep looking straight...";

status.style.color =
"#0074D9";
    if(!faceStartTime){
        faceStartTime = Date.now();
    }

    if(Date.now() - faceStartTime >= 2000){

        const canvas =
        document.createElement('canvas');

        canvas.width =
        video.videoWidth;

        canvas.height =
        video.videoHeight;
```

```
    canvas
    .getContext('2d')
    .drawImage(video,0,0);

    const capturedData =
    canvas.toDataURL('image/jpeg');

    photoInput.value =
    capturedData;

    document.getElementById("capturedImage").src =
    capturedData;

    document.getElementById("previewSection").style.display =
    "block";

    captured = true;

    status.innerText =
    "Photo Captured ✓";

    status.style.color =
    "green";

    checkSubmitStatus();
  }

  }else{

    faceStartTime = null;
    status.innerText =
    "Center your face and look straight";

    status.style.color =
    "#d93025";
```

```

    }
  }
});

/* =====
   CAMERA
   ===== */

navigator.mediaDevices
.getUserMedia({video:true})

.then(stream => {

  video.srcObject = stream;

  const loop = async () => {

    await faceMesh.send({
      image:video
    });

    requestAnimationFrame(loop);
  };

  loop();
})

.catch(err => {

  console.error(
    "Camera error access denied:",
    err
  );

  status.innerText =

```

```
"Camera error or Access Denied";
});

/* =====
FORM SUBMIT
===== */

document.querySelector("form")
.addEventListener("submit",function(e){

    if(!validateForm()){

        e.preventDefault();
        return;
    }

    const btn =
        document.getElementById("submitBtn");

    btn.innerText =
        "Processing...";

    btn.disabled = true;
});

// Calculate current quarter for dynamic UI rendering
const now = new Date();
const month = now.getMonth() + 1;
const year = now.getFullYear();
let quarterStr = "";
if(month >= 1 && month <= 3) quarterStr = "Q1 (January - March) - " + year;
else if(month >= 4 && month <= 6) quarterStr = "Q2 (April - June) - " + year;
else if(month >= 7 && month <= 9) quarterStr = "Q3 (July - September) - " +
year;
```

```
else quarterStr = "Q4 (October - December) - " + year;
document.getElementById("quarterText").innerText = quarterStr;
```

```
</script>
```

```
</body>
```

```
</html>
```

app.py

```
from flask import Flask, render_template, request, jsonify, redirect, url_for,
session
```

```
import sqlite3
```

```
import base64
```

```
import os
```

```
import face_recognition
```

```
from datetime import datetime
```

```
import re
```

```
import smtplib
```

```
import requests
```

```
from email.mime.text import MIMEText
```

```
from email.mime.multipart import MIMEMultipart
```

```
app = Flask(__name__)
```

```
app.secret_key = "security_key_aup_system"
```

```
UPLOAD_FOLDER = "static/photos"
```

```
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
```

```
# =====
```

```
# ADMIN PASSWORD
```

```
# =====
```



```
ADMIN_PASSWORD = "Itcsp@123"
```

```
# =====
```

```
# EMAIL SETTINGS
```

```
# =====
```

```
SENDER_EMAIL = "aupitcsp@gmail.com"
```

```
SENDER_PASSWORD = "zacsshoqhqbqzxi"
```

```
# =====
```

```
# DATABASE INIT
```

```
# =====
```

```
def init_db():
```

```
    conn = sqlite3.connect("employee.db")
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        CREATE TABLE IF NOT EXISTS employees (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            employee_id TEXT,
```

```
            name TEXT,
```

```
            jh_name TEXT,
```

```
            dmt_name TEXT,
```

```
            phone TEXT,
```

```
            email TEXT,
```

```
            photo_path TEXT,
```

```
            submission_time TEXT
```

```
        )
```

```
    """)
```

```
    conn.commit()
```

```
    conn.close()
```

```
init_db()
```

```
# =====
```

```
# EMAIL FUNCTION
```

```
# =====
```

```
def send_success_email(receiver_email, employee_name):
```

```
    try:
```

```
        subject = "AUP Registration Successful"
```

```
        body = f''''''
```

```
NOTE: This is an automated email. Please do not reply.
```

```
Hello {employee_name},
```

```
Your registration for DIGITALIZATION OF AUP & PASSWORD POLICY  
was completed successfully.
```

```
Your authentication and employee registration have been verified successfully.
```

```
Thank you.
```

```
Regards,
```

```
AUP Security System
```

```
''''''
```

```
msg = MIMEMultipart()
```

```
msg['From'] = SENDER_EMAIL
```

```
msg['To'] = receiver_email
```

```
msg['Subject'] = subject
```

```
msg.attach(MIMEText(body, 'plain'))
```

```
server = smtplib.SMTP('smtp.gmail.com', 587)
```

```
server.starttls()
```

```
server.login(SENDER_EMAIL, SENDER_PASSWORD)
```

```
server.sendmail(  
    SENDER_EMAIL,  
    receiver_email,  
    msg.as_string()  
)
```

```
server.quit()
```

```
print("EMAIL SENT SUCCESSFULLY")
```

```
# =====
```

```
# AUTO QUARTER SYSTEM
```

```
# =====
```

```
now = datetime.now()
```

```
month = now.month
```

```
year = now.year
```

```
if month >= 1 and month <= 3:
```

```
    current_quarter = "Q1"
```

```
    quarter_period = "January - March"
```

```
    next_quarter = "Q2"
```

```
    next_start = f"April 1, {year}"
```

```
elif month >= 4 and month <= 6:
    current_quarter = "Q2"
    quarter_period = "April - June"
    next_quarter = "Q3"
    next_start = f"July 1, {year}"
```

```
elif month >= 7 and month <= 9:
    current_quarter = "Q3"
    quarter_period = "July - September"
    next_quarter = "Q4"
    next_start = f"October 1, {year}"
```

```
else:
    current_quarter = "Q4"
    quarter_period = "October - December"
    next_quarter = "Q1"
    next_start = f"January 1, {year + 1}"
```

```
# =====
# SECOND EMAIL
# =====
```

```
policy_subject = f"AUP & Password Policy Update - {current_quarter}"
```

```
policy_body = f"""
```

```
Hello {employee_name},
```

```
This email contains the currently active AUP & Password Policies.
```

```
Current Quarter:
```

```
{current_quarter} ({quarter_period}) - {year}
```

```
Next Quarter:
```

{next_quarter}

Next Policy Activation Date:

{next_start}

Important Employee Instructions:

- Employees must update passwords regularly.
- Password sharing is prohibited.
- Face authentication is mandatory.
- Employees must follow cybersecurity rules.
- Employees must review updated policies every quarter.

Regards,

AUP Security System

```
policy_msg = MIMEMultipart()
policy_msg['From'] = SENDER_EMAIL
policy_msg['To'] = receiver_email
policy_msg['Subject'] = policy_subject
```

```
policy_msg.attach(MIMEText(policy_body, 'plain'))
```

```
server2 = smtplib.SMTP('smtp.gmail.com', 587)
server2.starttls()
server2.login(SENDER_EMAIL, SENDER_PASSWORD)
server2.sendmail(
    SENDER_EMAIL,
    receiver_email,
    policy_msg.as_string()
)
```

```
server2.quit()
```

```
print("SECOND EMAIL SENT")
```

```
except Exception as e:
```

```
    print("EMAIL ERROR:", e)
```

```
# =====
```

```
# SMS FUNCTION
```

```
# =====
```

```
def send_sms(phone, employee_name):
```

```
    message = f"""
```

```
Hello {employee_name},
```

```
Your registration for DIGITALIZATION OF AUP & PASSWORD POLICY  
was completed successfully.
```

```
Regards,
```

```
AUP Security System
```

```
"""
```

```
try:
```

```
    response = requests.post(
```

```
        'https://textbelt.com/text',
```

```
        {
```

```
            'phone': phone,
```

```
            'message': message,
```

```
            'key': 'textbelt'
```

```
        }
```

```
    )
```

```
    print(response.json())
```

```
except Exception as e:
```

```
    print("SMS ERROR:", e)
```

```
# =====
# ROUTES
# =====

@app.route("/")
def index():
    return render_template("main.html")

@app.route("/selection")
def selection():
    return render_template("selection.html")

@app.route("/user_options")
def user_options():
    return render_template("user_options.html")

# =====
# ADMIN LOGIN
# =====

@app.route("/admin_login", methods=["GET", "POST"])
def admin_login():

    if request.method == "POST":

        password = request.form.get("password")

        if password == ADMIN_PASSWORD:

            session['admin_logged_in'] = True

            return redirect(url_for("admin_dashboard"))

        return "<h1>Access Denied</h1>"
```

```

    return render_template("admin_login.html")

# =====
# ADMIN DASHBOARD
# =====

@app.route("/admin_dashboard")
def admin_dashboard():

    if not session.get('admin_logged_in'):
        return redirect(url_for("admin_login"))

    conn = sqlite3.connect("employee.db")
    conn.row_factory = sqlite3.Row

    cursor = conn.cursor()

    cursor.execute("SELECT * FROM employees ORDER BY id DESC")

    employees = cursor.fetchall()

    conn.close()

    return render_template("admin_dashboard.html", employees=employees)

# =====
# LOGOUT
# =====

@app.route("/admin_logout")
def admin_logout():

    session.pop('admin_logged_in', None)

```



```

    return redirect(url_for("index"))

# =====
# USER PAGES
# =====

@app.route("/register")
def register():
    return render_template("view.html")

@app.route("/face_login")
def face_login():
    return render_template("face_login.html")

# =====
# FORM SUBMISSION
# =====

@app.route("/submit", methods=["POST"])
def submit():

    employee_id = request.form.get("employee_id").strip()
    name = request.form.get("name").strip()
    jh_dmt = request.form.get("jh_dmt_name").strip()
    manual_input = request.form.get("jh_dmt_manual")

    if jh_dmt == "OTHER" and manual_input:
        jh_dmt = manual_input.strip()

    phone = request.form.get("phone").strip()
    email = request.form.get("email").strip()
    photo_data = request.form.get("photo_data")

    if len(employee_id) < 3:
        return "Invalid Employee ID"

```

```
if not re.match(r"^[A-Za-z ]{3,50}$", name):  
    return "Invalid Name"
```

```
if len(jh_dmt) < 2:  
    return "Invalid JH/DMT Name"
```

```
if not re.match(r"^[6-9]\d{9}$", phone):  
    return "Invalid Phone"
```

```
if not re.match(r"^[^\s@]+@[^\s@]+\.[^\s@]+$", email):  
    return "Invalid Email"
```

```
photo_path = ""
```

```
if photo_data:
```

```
    header, encoded = photo_data.split(",", 1)  
    image_data = base64.b64decode(encoded)
```

```
    filename =  
f"{employee_id}_{datetime.now().strftime('%Y%m%d%H%M%S')}.jpg"  
    photo_path = os.path.join(UPLOAD_FOLDER, filename)
```

```
    with open(photo_path, "wb") as f:  
        f.write(image_data)
```

```
conn = sqlite3.connect("employee.db")  
cursor = conn.cursor()
```

```
cursor.execute("""  
    INSERT INTO employees  
    (employee_id, name, jh_name, dmt_name, phone, email, photo_path,  
    submission_time)  
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
```

```
""", (
    employee_id,
    name,
    jh_dmt,
    jh_dmt,
    phone,
    email,
    photo_path,
    datetime.now().strftime("%Y-%m-%d %H:%M:%S")
))
```

```
conn.commit()
conn.close()
```

```
send_success_email(email, name)
send_sms(phone, name)
```

```
return render_template("success.html", name=name)
```

```
# =====
# REGISTERED PROFILE PHOTO LOADING
# =====
```

```
@app.route("/load_registered_photo", methods=["POST"])
def load_registered_photo():
```

```
    data = request.json
    employee_id = data.get("employee_id", "").strip()
    name = data.get("name", "").strip()
```

```
    if not re.match(r"^\d{5}$", employee_id) or len(name) < 3:
        return jsonify({"status": "error", "message": "Enter valid Employee ID
and name."})
```

```

conn = sqlite3.connect("employee.db")
cursor = conn.cursor()
cursor.execute(
    "SELECT photo_path FROM employees WHERE employee_id = ? AND
name = ? ORDER BY id DESC LIMIT 1",
    (employee_id, name)
)
row = cursor.fetchone()
conn.close()

if not row or not row[0] or not os.path.exists(row[0]):
    return jsonify({"status": "error", "message": "Registered photo not found
for this profile."})

with open(row[0], "rb") as f:
    encoded = base64.b64encode(f.read()).decode("utf-8")

return jsonify({
    "status": "success",
    "message": "Registered profile loaded.",
    "photo_data": f"data:image/jpeg;base64,{encoded}"
})

# =====
# FACE VERIFICATION
# =====

@app.route("/verify", methods=["POST"])
def verify():

    data = request.json
    mode = data.get("mode", "live")
    employee_id = data.get("employee_id", "").strip()

```

```

name = data.get("name", "").strip()

if mode == "registered":

    if not re.match(r"^\d{5}$", employee_id) or len(name) < 3:
        return jsonify({"status": "error", "message": "Invalid Employee ID or
name."})

    conn = sqlite3.connect("employee.db")
    cursor = conn.cursor()
    cursor.execute(
        "SELECT photo_path FROM employees WHERE employee_id = ?
AND name = ? ORDER BY id DESC LIMIT 1",
        (employee_id, name)
    )
    row = cursor.fetchone()
    conn.close()

    if row and row[0] and os.path.exists(row[0]):
        return jsonify({
            "status": "success",
            "message": f"Welcome back, {name}! Registered profile matched."
        })

    return jsonify({"status": "error", "message": "Registered profile not
found."})

photo_data = data.get("photo_data")

if not photo_data:
    return jsonify({"status": "error", "message": "No live photo captured."})

header, encoded = photo_data.split(",", 1)
live_blob = base64.b64decode(encoded)

```

```
with open("temp_login.jpg", "wb") as f:  
    f.write(live_blob)
```

```
live_img = face_recognition.load_image_file("temp_login.jpg")  
live_encs = face_recognition.face_encodings(live_img)
```

```
if not live_encs:  
    return jsonify({"status": "error", "message": "No face detected."})
```

```
conn = sqlite3.connect("employee.db")  
cursor = conn.cursor()  
cursor.execute("SELECT name, photo_path FROM employees")  
rows = cursor.fetchall()  
conn.close()
```

```
for name, path in rows:
```

```
    if path and os.path.exists(path):
```

```
        db_img = face_recognition.load_image_file(path)  
        db_encs = face_recognition.face_encodings(db_img)
```

```
        if db_encs and face_recognition.compare_faces([db_encs[0]],  
live_encs[0])[0]:
```

```
            return jsonify(  
                "status": "success",  
                "message": f"Welcome, {name}!"  
            )
```

```
    return jsonify({"status": "error", "message": "Verification Failed."})
```

```
# =====  
# RUN APP  
# =====
```

```
if __name__ == "__main__":
    app.run(debug=True)
camera.py
# Camera Route
@app.route('/camera')
def camera():

    camera = cv2.VideoCapture(0, cv2.CAP_DSHOW)

    while True:

        success, frame = camera.read()

        # Check if frame loaded properly
        if not success:
            continue

        cv2.imshow("Employee Camera", frame)

        key = cv2.waitKey(1)

        # Press S to save
        if key == ord('s'):

            cv2.imwrite("employee_photo.jpg", frame)

            break

    camera.release()
    cv2.destroyAllWindows()
    return "Photo Captured Successfully"
```

face_auth.py

```
import face_recognition
import cv2
import sqlite3
import os

# =====
# LOAD DATABASE IMAGES
# =====
conn = sqlite3.connect("employee.db")
cursor = conn.cursor()
cursor.execute("SELECT name, photo_path FROM employees")
users = cursor.fetchall()
known_encodings = []
known_names = []

for user in users:

    name = user[0]
    image_path = user[1]

    if os.path.exists(image_path):

        image = face_recognition.load_image_file(image_path)

        encodings = face_recognition.face_encodings(image)

        if len(encodings) > 0:

            known_encodings.append(encodings[0])
            known_names.append(name)

conn.close()

print("Database faces loaded successfully")
```



```
# =====  
# OPEN CAMERA  
# =====  
video_capture = cv2.VideoCapture(0)  
  
if not video_capture.isOpened():  
    print("Camera not opening")  
    exit()  
  
print("Camera started successfully")  
while True:  
  
    ret, frame = video_capture.read()  
  
    if not ret:  
        print("Failed to capture frame")  
        break  
  
    rgb_frame = frame[:, :, :-1]  
  
    face_locations = face_recognition.face_locations(rgb_frame)  
  
    face_encodings = face_recognition.face_encodings(  
        rgb_frame,  
        face_locations  
    )  
  
    for face_encoding in face_encodings:  
  
        matches = face_recognition.compare_faces(  
            known_encodings,  
            face_encoding  
        )
```

```
name = "Face Not Found"
```

```
if True in matches:
```

```
    first_match_index = matches.index(True)
```

```
    name = known_names[first_match_index]
```

```
    cv2.putText(  
        frame,  
        f"Attendance Marked: {name}",  
        (20,50),  
        cv2.FONT_HERSHEY_SIMPLEX,  
        1,  
        (0,255,0),  
        2  
    )
```

```
else:
```

```
    cv2.putText(  
        frame,  
        "Face Not Found",  
        (20,50),  
        cv2.FONT_HERSHEY_SIMPLEX,  
        1,  
        (0,0,255),  
        2  
    )
```

```
cv2.imshow("Face Authentication", frame)
```

```
# PRESS Q TO EXIT
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
video_capture.release()
```

```
cv2.destroyAllWindows()
```

EXPLANATION OF CODE

Flask Application Setup

The project is developed using the Flask framework, which handles routing, request processing, session management, and communication between the frontend and backend.

Database Connectivity

SQLite is used as the database for storing user details, facial information, attendance records, and authentication data. Database operations such as insertion, retrieval, and updates are performed through SQL queries.

User Registration Module

This module allows new users to register by entering their personal details. The webcam captures the user's facial image, which is processed and stored for future authentication.

Face Recognition Module

The Face Recognition library extracts facial features from captured images and generates facial encodings. These encodings are used to identify and verify users during login.

Liveness Detection Module

MediaPipe Face Mesh is used to detect facial landmarks and verify user movements such as head turns. This prevents spoofing attempts using photographs or videos.

Login Authentication Module

During login, the system captures a live facial image and compares it with stored facial encodings. Access is granted only when a valid match is found.

Attendance Management Module

After successful authentication, attendance is automatically recorded with the user's details, date, and time. The module ensures accurate attendance tracking.

Session Management

Flask sessions are used to maintain user login status and control access to protected pages until the user logs out.

Email Verification and Notifications

The system uses email services to send verification messages, alerts, or notifications to users when required.

Frontend Interface

HTML, CSS, and JavaScript are used to create an interactive user interface. JavaScript manages camera access, face capture, and communication with backend services.

Image Processing and Storage

Captured facial images are converted into suitable formats, processed for recognition, and stored securely in the system for future authentication.

Admin Dashboard

The dashboard provides administrators with access to user information, attendance records, and system monitoring features.

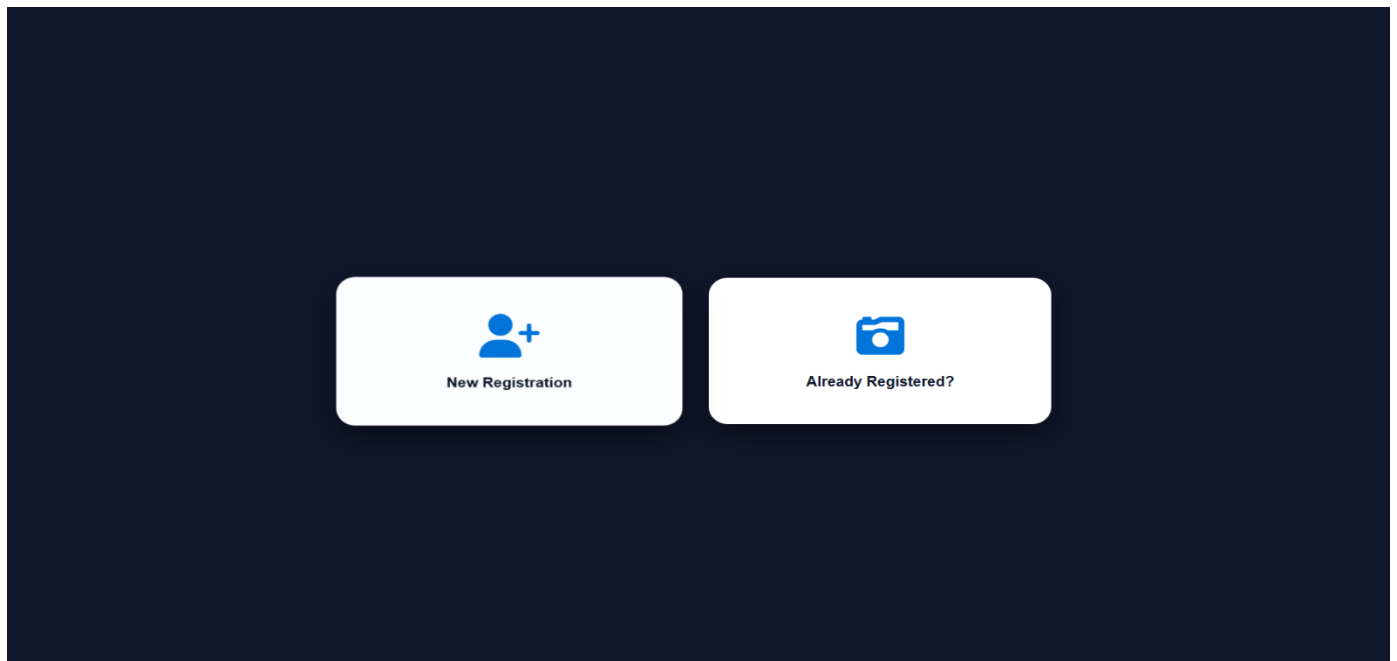
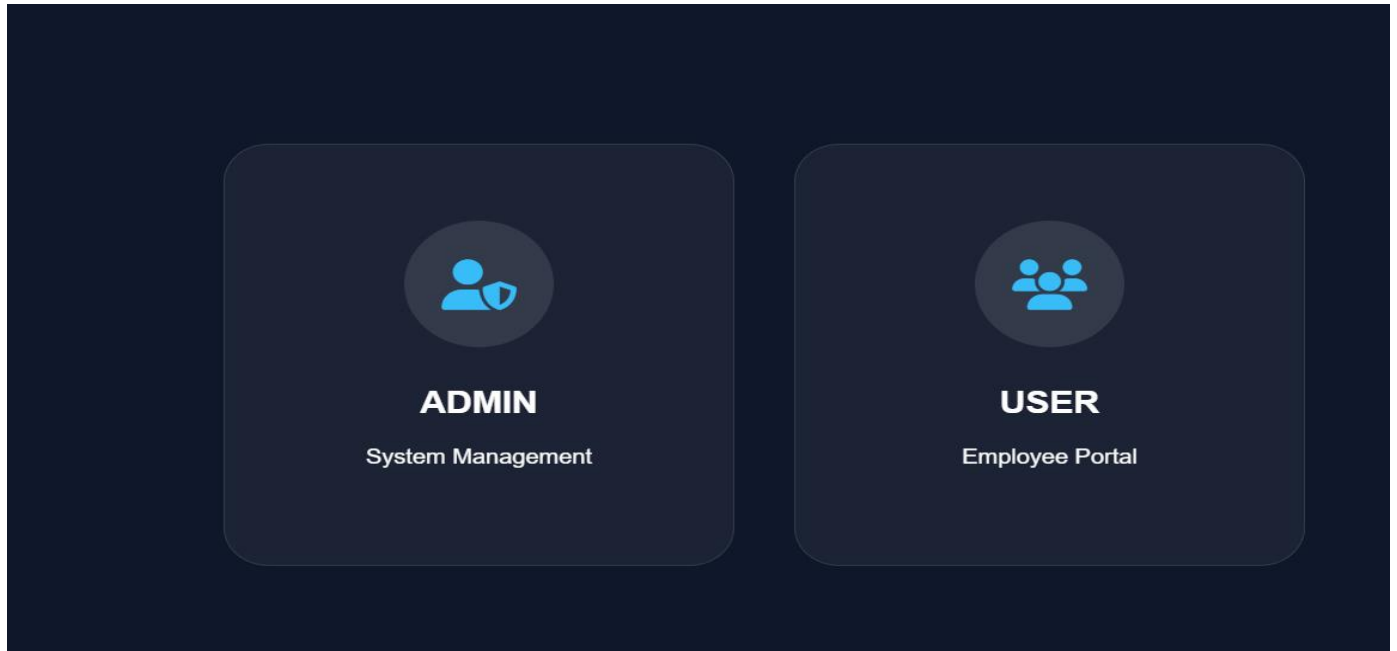
Security Features

The system combines face recognition, liveness detection, session handling, and secure data storage to enhance authentication security and prevent unauthorized access.

Logout Module

The logout functionality ends the user's session securely and redirects the user to the login page, ensuring system safety.

SCREENSHOTS:



Admin Dashboard

AUP & Password Policy Digitalization System

[Logout](#)

Total Employees: 35

Visible: 35

Attention Required (Poor/Average Ratings): 0

Search by Name / ID / Phone / Feedback

All Employees

All Ratings

Print Selected

Photo	Name	Employee ID	JH/DMT	Phone	Date	System Rating	Improvement Category	User Comments
	Ashwini	23521	DMT MANAGER	9010740669	2026-05-29 09:00:29	N/A	None	No comments left.
	Lohitha	90107	JH ISI	9866326816	2026-05-26 17:57:38	N/A	None	No comments left.
	Lohitha	901074	JH ISI	9866326816	2026-05-26 17:55:53	N/A	None	No comments left.
	Sathwika	879655	JH OTHERS	9010740669	2026-05-26 16:35:05	N/A	None	No comments left.
	Sathwika	879655	DMT TIME OFFICE	9010740669	2026-05-26 12:05:41	N/A	None	No comments left.
	Swathi	89796	JH ISI	8639648050	2026-05-25 18:53:34	N/A	None	No comments left.



AUP & Password Policy

Digitalization System

Current Active Policies

Password Policy: Version 3.2

AUP Policy: Q2 (April - June) - 2026

Employees must accept latest policies before authentication.

⚠ Next AUP & Password Policy revision scheduled for October 2026

Employee ID *

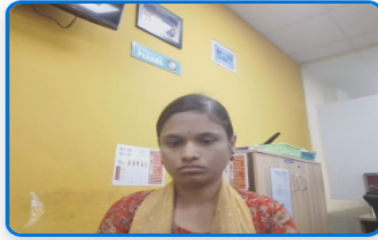
85444

Full Name *

pratyisha

JH / DMT Category *

DMT MANAGER



Preview Photo

Recapture Photo

System & Registration Feedback

Rate Your Experience *

Very Good



Area of Improvement

None (Perfect)



Comments / Issues Faced

no

Mandatory Security Policies *

1. Employee must maintain confidentiality of company data.
2. Sharing passwords with others is strictly prohibited.
3. Unauthorized system access is not permitted.
4. Face authentication is mandatory for verification.
5. Employees must follow AUP security standards.
6. Fake registrations will lead to access denial.



I agree to all mandatory protocols and security policies

REGISTER NOW

System Feedback Loop

Overall Rating

Excellent



System Category Improvements

No Suggestions



Detailed User Feedback Comments

Type your creative suggestions or tracking portal experience notes here...



Welcome Aboard

Registration Completed Successfully



Email Verified Successfully

Face Match Result

96%



CONCLUSION

The Face Recognition Attendance and Authentication System successfully provide a secure, efficient, and automated solution for user verification and attendance management. By integrating face recognition technology with liveness detection mechanisms, the system ensures that only genuine users can access the platform, thereby reducing the risk of unauthorized access and identity spoofing. The use of facial biometrics eliminates the need for traditional manual attendance methods and enhances overall security.

The system streamlines the registration and authentication process by combining facial data capture, OTP verification, policy acknowledgment, and attendance recording within a single platform. The implementation of liveness detection through facial movement verification further strengthens the reliability of the authentication process. Additionally, the use of Flask, SQLite, Face Recognition, and MediaPipe technologies enables the system to operate effectively while maintaining simplicity and ease of deployment.

The developed application demonstrates how modern biometric technologies can be utilized to improve organizational security and user convenience. The system maintains accurate attendance records, reduces manual effort, and provides a user-friendly interface for both employees and administrators. Overall, the project achieves its objectives of creating a secure, reliable, and automated attendance management solution that can be adapted and enhanced for real-world organizational environments.

Another important feature of the project is the collection of employee feedback during registration. This functionality helps the organization understand user experiences, identify potential issues, and continuously improve the registration process. The digital storage of records also makes policy tracking, auditing, and future reference easier and more reliable than traditional paper-based methods.

The project demonstrates the practical application of web technologies, database management, authentication mechanisms, and security principles in solving real-world organizational challenges. The successful integration of these components has resulted in a robust and scalable solution that meets the current requirements of digital policy management and employee verification. In the future, the system can be enhanced with additional features such as advanced facial recognition, automated report generation, analytics dashboards, role-based access control, and cloud-based data storage. These improvements can further increase the efficiency, scalability, and security of the application.

Overall, the AUP & Password Policy Digitalization System provides a reliable, secure, and user-friendly solution for managing employee registrations and policy acceptance. The project successfully achieves its objectives and demonstrates how digital transformation can simplify administrative processes while strengthening organizational security and compliance.

THANK YOU